

АННОТАЦИЯ

Тема выпускной квалификационной работы — «Разработка платформы для генерации резюме с применением технологий искусственного интеллекта на основе стартапа Resugen», в рамках которой были рассмотрены современные тенденции цифровизации кадрового делопроизводства и внедрение алгоритмических систем отбора персонала (Applicant Tracking Systems).

Целью работы является создание архитектуры и минимально жизнеспособного продукта (MVP) платформы «Resugen», выступающей в роли цифрового карьерного ассистента и обеспечивающей повышение совместимости резюме с алгоритмами машинного анализа. Для достижения поставленной цели были сформулированы основные задачи:

1. Провести комплексный анализ международного и российского рынков сервисов для создания и оптимизации резюме, провести анализ аудитории.
2. Сформулировать детализированные функциональные и нефункциональные требования к разрабатываемой платформе.
3. Спроектировать архитектуру сервисов бизнес-процесса, аналитики и фронтенд часть.
4. Реализовать сервис бизнес-процесса, аналитики и фронтенд часть .
5. Описать жизнеспособную бизнес-модель проекта, построить финансовую модель, оценить рыночный потенциал (TAM, SAM, SOM) и проанализировать риски.

Работа состоит из 57 страниц, включающих 11 рисунков, 1 таблицу. Для решения поставленных задач использовано 30 источников. Область применения данной работы — техническое обеспечение процессов трудоустройства.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	2
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ИНТЕЛЛЕКТУАЛЬ- НЫХ HR-СЕРВИСОВ.....	5
1.1 Эволюция систем автоматизации рекрутмента и анализа резюме.....	5
1.2 Современные методы обработки естественного языка в HR-задачах	8
1.3. Проблематика и требования интеллектуальных сервисов поддерж- ки при создании резюме.....	11
1.4 Обзор и классификация существующих решений.....	12
2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СЕРВИСОВ БИЗНЕС-ПРО- ЦЕССА, АНАЛИТИКИ И ФРОНТЕНДА.....	16
2.1 Функциональные и нефункциональные требования к разрабатывае- мой платформе.....	16
2.2 Проектирование архитектуры сервисов бизнес-процесса и аналити- ки.....	18
2.3 Реализация сервисов бизнес-процесса.....	23
2.5 Реализация фронтенд-части и интеграция с сервисами.....	31
3 ЭКОНОМИЧЕСКАЯ КОМПОНЕНТА И ТЕСТИРОВАНИЕ СТАР- ТАП-ПРОЕКТА.....	37
3.1 Тестирование MVP.....	37
3.2 Оценка эффективности платформы методом ансамблевого LLM- скоринга.....	42
3.3 Анализ сильных и слабых сторон.....	46
3.4 Оценка рыночного потенциала.....	48
ЗАКЛЮЧЕНИЕ.....	53
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	54

ВВЕДЕНИЕ

На сегодняшний день информационные технологии развиваются с огромной скоростью и проникают во все сферы деятельности человека. Процессы, которые ранее требовали значительных временных и человеческих ресурсов, все чаще автоматизируются с помощью программных алгоритмов и систем искусственного интеллекта. Сфера управления персоналом и рекрутмента не является исключением: на смену ручному отбору и субъективной оценке приходят автоматизированные системы подбора, способные в кратчайшие сроки обрабатывать большие объемы данных.

Под влиянием цифровизации изменились и требования к первичному взаимодействию между соискателем и работодателем. Если раньше резюме оценивалось исключительно человеком, то теперь на первом этапе оно зачастую проходит через фильтры алгоритмических систем отбора. Эти системы анализируют структуру документа, наличие определенных ключевых элементов и семантическое соответствие описанию вакансии. В результате качество трудоустройства соискателя начинает напрямую зависеть не только от его профессиональных компетенций, но и от того, насколько корректно его резюме подготовлено с технической точки зрения для машинного анализа.

Исходя из этого, возникает объективная потребность в инструментах, которые помогают соискателю адаптироваться к новым технологическим реалиям рынка труда. Пользователю необходим сервис, способный не просто предоставить шаблон для заполнения, но и выступить в роли интеллектуального ассистента: проанализировать требования вакансии, дать рекомендации по улучшению содержания резюме и спрогнозировать его совместимость с автоматизированными системами отбора. Разработка подобных сервисов требует применения современных методов обработки естественного языка и проектирования сложных распределенных систем.

Актуальность работы обусловлена возрастающим разрывом между традиционными методами составления резюме и современными

алгоритмическими подходами к их оценке. В условиях постоянного совершенствования технологий автоматизации рекрутинга, умение создавать конкурентоспособное и технологически совместимое резюме становится критически важным навыком. В связи с этим возникает необходимость в создании доступных и эффективных цифровых инструментов, способных снизить порог вхождения соискателей в цифровую среду кадрового отбора и повысить прозрачность этого процесса.

Целью работы является разработка архитектуры и MVP для интеллектуальной веб-платформы «Resugen», выступающей в роли «цифрового карьерного ассистента»

Для достижения поставленной цели необходимо решить следующие основные задачи работы:

1. Провести комплексный анализ международного и российского рынков сервисов для создания и оптимизации резюме, провести анализ аудитории.
2. Сформулировать детализированные функциональные и нефункциональные требования к разрабатываемой платформе.
3. Спроектировать архитектуру сервисов бизнес-процесса, аналитики и фронтенд часть.
4. Реализовать сервис бизнес-процесса, аналитики и фронтенд часть .
5. Описать жизнеспособную бизнес-модель проекта, построить финансовую модель, оценить рыночный потенциал (TAM, SAM, SOM) и проанализировать риски.

Объектом исследования являются системы генерации структурированного контента.

Предметом исследования является автоматизация рекрутмента и создания резюме с использованием языковых моделей.

Новизна работы заключается в комплексном подходе к проектированию платформы, который сочетает в себе детерминированные алгоритмы структурной оптимизации документа с методами семантического анализа и

генерации текста. В отличие от упрощённых конструкторов документов, предлагаемое решение ориентировано на обеспечение совместимости резюме с алгоритмическими механизмами отбора за счёт глубокой интеграции методов обработки естественного языка в процесс создания документа.

Практическая значимость работы заключается в создании готового к использованию программного инструмента, который позволяет конечным пользователям сократить временные затраты на подготовку профессиональных документов при трудоустройстве и повысить их релевантность требованиям конкретных вакансий. Разработанные в ходе работы архитектурные принципы и программные модули могут быть использованы в качестве основы для дальнейшего развития интеллектуальных ассистентов в области управления карьерой.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ИНТЕЛЛЕКТУАЛЬНЫХ HR-СЕРВИСОВ

1.1 Эволюция систем автоматизации рекрутмента и анализа резюме

Интеллектуальные HR-системы возникли из классических ATS, которые с 1970-х годов трансформировались от простых баз резюме до облачных и AI-решений[1][2]. Современные ATS автоматизируют 70–80% рутинных операций рекрутера (сбор резюме, рассылки, первичный отбор)[3][4]. Согласно отраслевой статистике, порядка 75% поступающих резюме отфильтровываются системами на первых этапах[3], что значительно ускоряет процесс найма. На Рисунке 1.1.1 показана хронология развития данных систем.

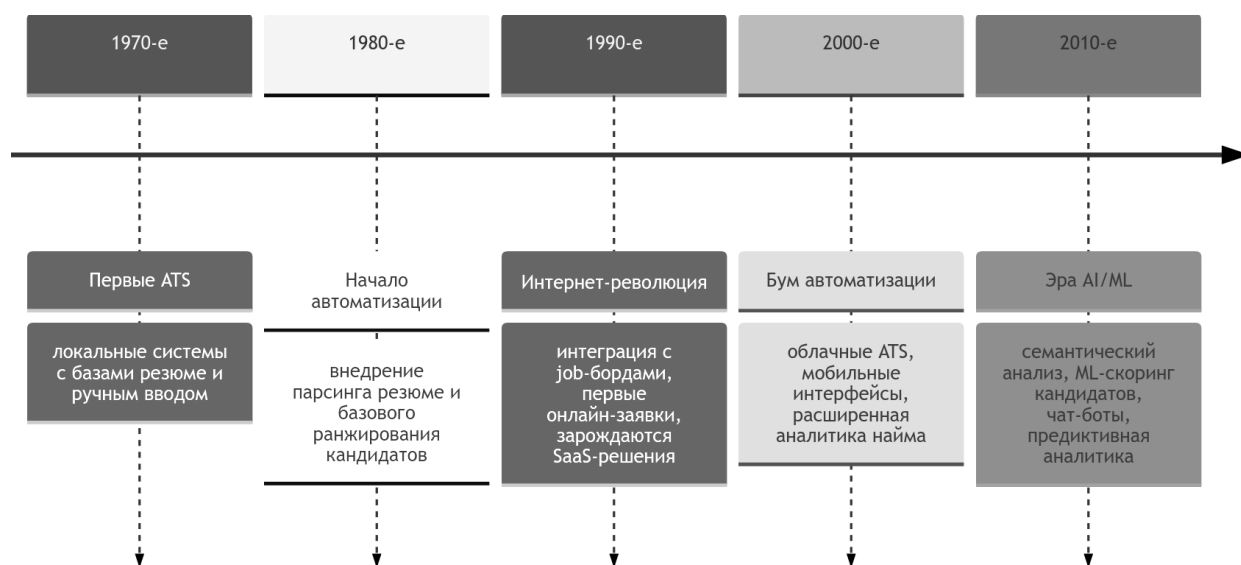


Рисунок 1.1.1 — Хронология развития ATS

Появление первых ATS датируется 1970-ми годами[1]. Эти системы представляли собой электронные хранилища резюме, где данные вводились вручную; поиск осуществлялся по простым ключевым словам. В 1980-х годах появились первые алгоритмы парсинга резюме[1] – системы автоматически выбирали из анкеты ключевые поля (образование, опыт) и сортировали кандидатов. Однако ранние ATS были громоздкими и дорогими: позволить их внедрение могли лишь крупные корпорации. В этот период системы

рекрутинга давали преимущество централизованной базе кандидатов, но чувствительность и гибкость фильтров оставляли желать лучшего.

Распространение интернета в 1990-х радикально изменило HR-инструменты[1]. Веб-сайты компаний и job-порталы (например, Monster, CareerBuilder) упростили публикацию вакансий и сбор резюме. ATS стали интегрироваться с этими ресурсами: кандидаты могли подавать заявки онлайн, а системы автоматически импортировали данные[1]. В 2000-х, благодаря облачным технологиям, ATS вышли за рамки enterprise: SaaS-модель обеспечила малым и средним предприятиям доступ к современным инструментам без капитальных затрат[1][3]. Появились мобильные приложения для рекрутеров и кандидатов, что ещё больше ускорило процесс найма. Автоматические алгоритмы анализировали сразу тысячи резюме, формируя шорт-листы по релевантности (с учётом опыта, навыков, образования)[5]. В результате время закрытия вакансий стало измеряться днями, а не неделями – показатель времени закрытия позиций существенно сократился[3][4].

Начиная с 2010-х, ATS активно используют машинное обучение и NLP[2][3]. Системы могут семантически сравнивать вакансии и резюме: например, модели BERT узнают, что «React-разработчик» и «Frontend React» эквивалентны[6]. Автоматические скоринг-функции оценивают каждого кандидата по сложным алгоритмам, учитывая десятки параметров (навыки, стаж, достижения), часто формируя единый балл согласованно с историей успешных наймов. Интеллектуальные инструменты также включают возможность прогнозирования успеха: на основе данных прошлых наймов система вычисляет вероятность успешного прохождения испытательного срока или долгосрочной эффективности кандидата. Расширенные ATS предлагают чат-ботов и виртуальных ассистентов, которые проводят предскрининг, задают первичные вопросы кандидатам и согласуют интервью[2][7]. При этом системы адаптируются к контексту и синонимам, анализируя не просто ключевые слова, а смыслы фраз.

Современные ATS имеют преимущества:

6. выполняют комплексную автоматизацию рутинных операций, таких как поиск и скрининг кандидатов, рассылка сообщений, планирование[3][4];
7. ведут объединённые базы данных кандидатов с историей взаимодействий;
8. предоставляют отчётность и аналитику в реальном времени по KPI найма (время на этап, источники заявок и так далее).

Это позволяет существенно повышать эффективность рекрутеров и формировать стратегию подбора на основе данных.

Однако данные системы имеют ограничения и не способны полностью заменить роль человека. Системы уязвимы к качеству исходных данных: ошибки парсинга документов из-за особенности их формата или нетривиального форматирования содержимого текста могут ухудшать результаты. Алгоритмы могут упустить нестандартные резюме и от выдающихся кандидатов, если они не соответствуют заданным настройкам. Кроме того, автоматические фильтры известны своей «жесткостью»: по статистике StandOut CV, около 75% резюме отсекаются автоматикой на первом этапе[3], что уменьшает время рекрутера, но требует контроля ошибок. Также важно учитывать этические аспекты: без должного надзора системы могут незаметно воспроизводить скрытую предвзятость в данных, включая даже гендерные, возрастные или иные смещения[2][6]. Следовательно, технологии ATS следует применять в тандеме с человеческим контролем и периодической валидацией.

1.2 Современные методы обработки естественного языка в HR-задачах

Современный рекрутмент опирается на достижения NLP для автоматизации анализа текстовой информации. Основные технологии включают распознавание сущностей, векторное встраивание текстов, семантический поиск, классификацию, а также глубокие нейронные сети (Transformer, RNN) и семантические онтологии навыков. Эти методы применяются для извлечения и сравнения информации в резюме и вакансиях, построения рекомендательных систем и интерактивных чат-ботов.

Ключевые поля из резюме и вакансий (имя, опыт, образование, навыки) выделяются методами NER (Named Entity Recognition) и синтаксического анализа. Например, современные модели умеют корректно распознавать дату и промежутки времени, наименование компаний и должностей в произвольном формате[6]. Для этого используют как классические CRF-модели, так и нейросетевые подходы (BERT/BiLSTM, spaCy, DeepPavlov и другие). На их основе формируют расширенные графы знаний: связывают кандидата с индустрией, описывают иерархию навыков. Результат – структурированная база данных из резюме и вакансий, пригодная для дальнейшего анализа.

При подборе кандидатов системы переходят от точного совпадения слов к семантической близости. Словесные представления (word2vec, fastText, SBERT) преобразуют текстовые описания в векторы. Обученные на HR-корпусах трансформеры (BERT и производные) учитывают контекст: они понимают, что «project manager» и «руководитель проектов» – синонимичные термины[6]. Это позволяет ранжировать отклики не только по вхождению ключевых слов, но и по смысловой релевантности. NLP-алгоритмы ранжирования (learning-to-rank) обучают модель отличать успешные совпадения резюме с заданной вакансией. В качестве тренировочных данных используются прошлые решения по найму (отмечено в ACL-обзоре как перспективное

направление[2]). Результат – многофакторный скоринг кандидатов, улучшающий качество отбора.

Помимо простого фильтра, современные системы применяют машинное обучение для классификации резюме по релевантности или прогнозирования поведения кандидата. Например, модели классифицируют резюме как «соответствующее» или «не соответствующее» требованиям вакансии. Также используются регрессионные алгоритмы для прогноза долгосрочности контракта и уровня успеха кандидата[2]. Часто решения представляют NLP-задачу как ранжирование: алгоритм обучается на помеченных данных – кто был приглашён на интервью, нанят или уволен – и находит оптимальные критерии оценки. Такие подходы ускоряют первичный отбор, позволяя HR-специалистам сосредоточиться на узком круге сильных кандидатов.

NLP лежит в основе интерактивных HR-ботов, которые общаются с кандидатами при помощи текста или речи. Например, чат-боты задают уточняющие вопросы соискателю, помогают заполнить форму отклика или записаться на интервью. Алгоритмы Intent Detection и Dialog Management обеспечивают понимание вопросов («Какие требования к опыту?») и поиск ответов в базе вакансий. Некоторые системы интегрированы с ASR (Automatic Speech Recognition) и TTS (Text-To-Speech), позволяя провести голосовое интервью с анализом речевых характеристик. По оценкам HR-отрасли, чат-боты ускоряют время ответа кандидату и сокращают операционные издержки на 30–50%[8], особенно при массовом найме.

Современные ATS-платформы начинают использовать LLM (Large Language Models) и генеративный ИИ: они автоматически составляют тексты вакансий, индивидуальные ответы кандидатам и даже рекомендации по улучшению резюме. Например, система может предложить переформулировать описание достижений кандидата под ключевые требования. Гибридные решения комбинируют традиционные алгоритмы с нейросетями: параллельно с BERT-классификатором вычисляют числовые метрики совпадений и

объединяют их в ансамбль. Такие сложные конвейеры улучшают точность отбора и минимизируют влияние брешей в обучении моделей.

Преимущества методов NLP:

- обеспечение масштабируемости (тысячи резюме обрабатываются за секунды);
- объективность (снижая влияние субъективности при первичном скрининге[4][8]) и возможность глубокой аналитики (выявление скрытых шаблонов успешности);
- обработка естественного языка превращает рекрутинг в Data-driven процесс, позволяя анализировать эффективность каналов поиска и качество оценок.

Тем не менее, методы NLP также ограничены, поскольку модели требовательны к данным: ошибки при извлечении (например, из PDF или изображения) оказывают значительное влияние на результат. Автоматические системы являются лишь инструментом поддержки решений, который может пропустить нетривиальных кандидатов или повторить предвзятость данных[4][6]. Требуется регулярная калибровка моделей и участие специалистов, контролирующих работу системы.

Можно сделать вывод, что систематическое применение NLP в рекрументе стремительно расширяет возможности автоматизации данной сферы. Распределение задач между машинами и людьми становится более чётким: человек оставляет за собой стратегические решения и валидацию результатов[9]. При этом важно помнить, что внедрение ИИ-технологий требует тщательной подготовки данных и постоянного мониторинга их работы[10]. Интегрированный подход, сочетающий современные NLP-алгоритмы и экспертное знание бизнеса, позволяет создавать интеллектуальные HR-сервисы, способные ускорить и улучшить подбор персонала.

1.3. Проблематика и требования интеллектуальных сервисов поддержки при создании резюме

Резюме просматривают не люди, а алгоритмы, ищущие точные совпадения ключевых слов и паттернов. Это приводит к нескольким проблемным аспектам:

- **Массовая автофильтрация резюме.** Более половины заявок блокируется «на первом этапе» без участия человека[11][12]. Кандидаты не получают обратной связи о поводах для отказа (строгие фильтры по опыту, образованию или определённым терминам[13][14]).
- **Нечеловеческие предпочтения и смещения.** Алгоритмы наследуют тренды обучающих данных. Например, известный кейс Amazon 2018 показал, что ИИ отказывал женщинам (из-за смещённого обучающего набора). В современном подборе риски «black-box» решений остаются: алгоритм может отбросить резюме с обучением на определённых курсах или русскими именами без объяснения. Это требует особого контроля над искусственным интеллектом[12].
- **Ошибки форматирования и контентные пробелы.** Сложные макеты (таблицы, графика, изображения) часто «ломают» разборщики резюме[14][15]. Аналогично, использование синонимов вместо строгих терминов вакансии снижает процент попадания: ATS обычно ищут точные ключевые фразы[14]. Требуется автоматическое выявление таких проблем: например, уведомлять кандидата об обнаруженных «странных» объектах и предлагать упростить формат (одно-колоночный текст, стандартные шрифты)[15].
- **Зависимость от AI и «нечеловеческая» речь.** Попытки полностью доверить написание CV генеративным моделям приводят к чрезмерно формальным или пафосным формулировкам[16][13]. Лексика выходит за привычный стиль (кандидаты используют фразы, которые люди редко говорят), что снижает доверие. С другой стороны, массовое

применение одинаковых шаблонов (например, автоматических сопроводительных писем) делает кандидатские профили однообразными[16].

- **Приватность и безопасность данных.** Резюме содержат личные данные (имена, контакты и другие), часто кандидаты загружают их в облако. Вопросы GDPR (для зарубежных компаний) и российского законодательства становятся критичными[17].

Учитывая вышеуказанные проблемы, «умный помощник» при создании резюме должен реализовывать следующий функционал:

- **Парсинг и анализ контента.** Распознавать сущности (имя, образование, опыт, навыки) с помощью NER и синтаксического анализа[18]. Автоматически выделять ключевые навыки в тексте. Проверять наличие и формат важных полей (например, даты в единообразном формате).
- **ATS-оптимизация.** Платформа должна анализировать описания вакансий и предлагать включить точные ключевые фразы из них в резюме[14]. Например, конвертировать синонимы или жаргон должности в формальные термины работодателя. Это может включать дообучение модели на локальных данных компании (fine-tuning) для лучшего понимания внутренней терминологии.
- **Улучшение текста.** Сервис выступает редактором: предлагает переформулировки для ясности и конкретики, убирает избыточные слова. Генерирует примеры достижений («увеличил продажи на X%») и добавляет релевантные метрики (например, количество подчинённых или объем проектов). Это делает резюме более информативным для алгоритмов и людей.
- **Интерактивные боты и подсказки.** Уточнение информации в интерактивных полях, вопросы про опыт, детали. Это снижает разрыв между формальным резюме и личным опытом.

1.4 Обзор и классификация существующих решений

Российский HR-рынок представлен сочетанием локальных ATS-систем, AI-конструкторов резюме и порталов. В Таблице 1.4.1 представлен список прямых и косвенных конкурентов.

Таблица 1.4.1 — Прямые и косвенные конкуренты

Продукт	Категория	Русский язык	AI-Функции	Модель
Huntflow	ATS/CRM	Да	Да	Подписка
Talantix	ATS/CRM	Да	Да	Индивидуальный план
СберПодбор	ATS/CRM	Да	Да	Freemium, от 5000 руб/месяц
CareerLab	AI-резюме	Нет	Да	Freemium
Rezi	AI-резюме	Нет	Да	
Resume.com	Конструктор	Нет	Нет	Бесплатно
Kickresume	AI-конструктор	Нет	Да	
Workday HCM	ATS/CRM	Нет	Да	Индивидуальный план
SAP SuccessFactors	ATS/CRM	Нет	Да	Индивидуальный план
Canva	Дизайн	Да	Да	Freemium
Teal	AI-резюме	Нет	Да	Бесплатно

Например, «ТалантЭкс» («Talantix») – ATS/CRM из экосистемы «HeadHunter» (глубокая интеграция с hh.ru и CV-парсером); «СберПодбор» – HR-платформа Сбера для массового найма с мобильным интерфейсом; «Поток» (Potok.io) – решение для рекрутинга с упором на массовый и госзаказ. [19] Эти системы оптимизированы под русскоязычные данные, предлагают стандартный функционал (автопарсинг резюме, аналитика откликов) и часто интегрируются с российскими порталами («HeadHunter», «SuperJob»)[20][17].

Среди сервисов генерации резюме и карьерных помощников: «GarmonyCV» – чат-бот-ассистент к «CareerLab» для интерактивного создания «резюме» (сам подбирает фразы, проверяет стиль). «ruGPT» – отечественная нейросеть, заменяющая западные LLM для генерации текстов. Большинство этих сервисов позиционируются как B2C, имеют freemium-

модель (базовый функционал бесплатен, расширенные возможности по подписке).

Крупнейшие площадки по трудоустройству включают встроенные резюме-редакторы. Например, «HeadHunter» бесплатно генерирует однотипное резюме по шаблону с возможностью выбора языка. «Работа.ру» предлагает обширный набор шаблонов и мультиязычный интерфейс.

Некоторые сервисы маскируются под HR-ассистентов. К примеру, «Headz» – приложение, где соискатели «ставят лайки» вакансии, а алгоритм находит лучшие соответствия по зарплате и стеку. При этом никакие личные данные не передаются – контакты раскрываются только после согласия обеих сторон. Во многих российских ATS появились встроенные чат-боты для первого интервью и сбора данных (см. «ИИ-агент для первого контакта» у HH[13]).

Глобальные HR-технологии делятся на корпоративные решения и потребительские AI-сервисы:

- **Корпоративные ATS/HCM.** Лидеры – «Workday», «SAP SuccessFactors» («Taleo»), «Oracle Cloud HCM», «iCIMS», «Greenhouse», «SmartRecruiters», «Bullhorn» и др. Эти платформы поддерживают многочисленные интеграции, аналитику и часто имеют функции на основе искусственного интеллекта (ранжирование кандидатов, предсказание эффективности). По оценкам аналитиков, мировой рынок ATS растёт ~7–10% в год[17], достигая к 2025 г. нескольких миллиардов долларов (консервативная оценка – \$2.47 млрд в 2025[17]). Большинство таких систем ориентированы на англоязычный сегмент, но предлагают мультиязычные интерфейсы.
- **Онлайн-конструкторы резюме.** «CareerLab» – «экосистема» для создания резюме с AI-анализом вакансий, адаптацией под каждую позицию, подсказками экспертов и гарантией ATS-совместимости[21]; В мировой практике популярны «Kickresume» – гибридный сервис с интеллектуальным редактором и большим количеством шаблонов[11];

«Resume.com» – бесплатный конструктор с сотнями шаблонов, ориентированный на рекрутеров из крупных корпораций[21]; «Enhancv» – креативный конструктор с продвинутыми подсказками (имеются ограничения бесплатного периода)[21]; «Rezi» – платформа, полностью автоматизирующая создание CV при помощи языковых моделей[21]. Также широко используются сервисы «Teal», «Jobscan», «MyPerfectResume», «VisualCV» и «Canva» (предлагает шаблоны, но без гарантии ATS-совместимости[11]). Многие из этих инструментов генерируют не только резюме, но и сопроводительные письма на основе введенных данных и актуальных вакансий.

Таким образом, российский рынок характеризуется сосуществованием специализированных локальных ATS/CRM-систем (Huntflow, Talantix, Сбер-Подбор), глубоко интегрированных с отечественными порталами по трудоустройству и оптимизированных под русскоязычный контент, и широкого спектра международных AI-сервисов для создания и адаптации резюме (CareerLab, Kickresume, Rezi и др.), при этом как глобальные корпоративные платформы (Workday, SAP), так и российские игроки активно внедряют искусственный интеллект для автоматизации рекрутинговых процессов, предлагая как B2B-подписки, так и freemium-модели для конечных соискателей.

2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СЕРВИСОВ БИЗНЕС-ПРОЦЕССА, АНАЛИТИКИ И ФРОНТЕНДА

2.1 Функциональные и нефункциональные требования к разрабатываемой платформе

Разрабатываемая платформа Resugen представляет собой комплексный веб-сервис, предназначенный для автоматизированной генерации и оптимизации резюме с учётом требований современных систем автоматического отбора. Основной функциональной задачей платформы является устранение разрыва между субъективными представлениями соискателей о качественном резюме и объективными алгоритмическими критериями, предъявляемыми к документам автоматизированными системами. Для достижения этой цели система должна обеспечивать управление пользователями, возможность создания и редактирования анкет, генерацию и хранение резюме, работу с вакансиями, а также предоставление аналитических инструментов для оценки качества создаваемых документов.

Система должна поддерживать полноценный цикл управления пользователями, включая регистрацию и авторизацию, с подтверждением учётной записи через электронную почту, а также предоставлять возможность восстановления доступа при утере пароля. Пользовательский аккаунт служит основой для последующей работы с анкетой, резюме и вакансиями, а также хранения истории изменений и аналитических отчётов.

Ключевым элементом платформы является работа с анкетами. Анкета формирует основу для генерации резюме и должна включать все необходимые данные о пользователе: персональные сведения, образование, опыт работы, профессиональные навыки и достижения. Платформа обеспечивает проверку полноты и корректности введённой информации, в том числе валидацию обязательных полей.

Процесс работы с резюме организован таким образом, чтобы пользователь мог получить документ, полностью оптимизированный под требования конкретной вакансии. Генерация резюме осуществляется на основе данных анкеты и предоставленного пользователем описания вакансии. Система автоматически формирует структуру документа, соответствующую стандартам ATS, обеспечивая корректное чтение и парсинг системой работодателя. Кроме того, для повышения релевантности текста резюме используется семантический анализ на основе векторных представлений и адаптация содержания с помощью больших языковых моделей. Такой подход обеспечивает не только техническую корректность, но и улучшает соответствие резюме ожиданиям работодателя, повышая вероятность прохождения первичного отбора.

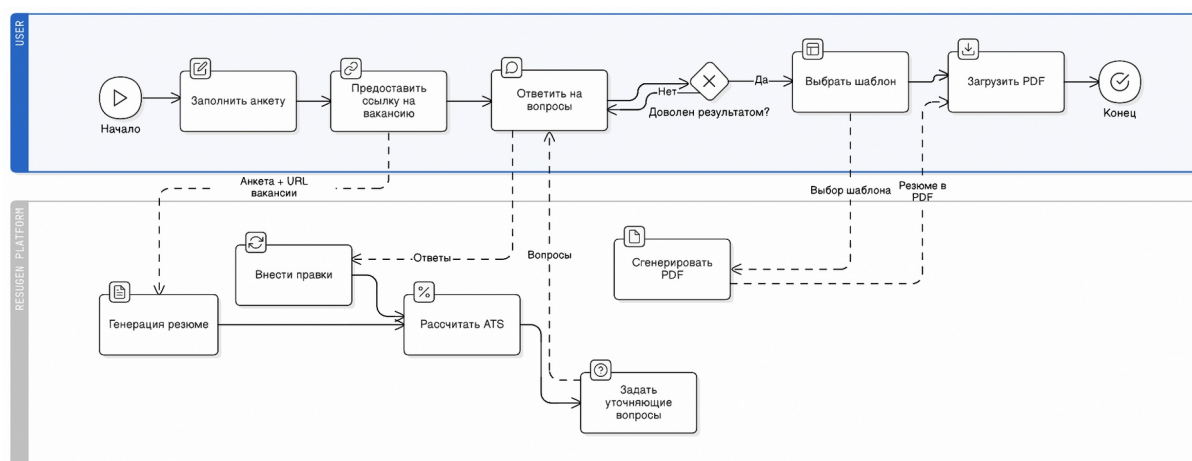


Рисунок 2.1.1 — Основой бизнес-процесс

Платформа также реализует возможности работы с вакансиями, включая импорт и сохранение вакансий из внешних источников, таких как hh.ru. Система извлекает ключевые требования и компетенции, указанные в вакансии, что позволяет автоматически адаптировать резюме под конкретную позицию. Пользователь получает возможность создавать, редактировать и удалять вакансии, хранить их в личном кабинете, а также повторно использовать для генерации новых резюме. Такая функциональность обеспечивает полное управление процессом подготовки документов и сокращает время на анализ требований.

Аналитическая составляющая платформы включает предоставление

подробных отчётов по каждому резюме. Пользователь получает ATS-score, отражающий соответствие документа требованиям системы автоматического отбора, а также выявление сильных и слабых сторон текста. Сервис генерирует конкретные рекомендации по улучшению содержания и структуры резюме, обеспечивая прозрачность процесса и предоставляя пользователю полный контроль над финальным документом. Аналитические данные представлены в удобном визуальном формате, что позволяет быстро ориентироваться в результатах и принимать решения о доработке резюме.

2.2 Проектирование архитектуры сервисов бизнес-процесса и аналитики

Разработка платформы Resugen, предназначенной для автоматизированной генерации и оптимизации резюме, требует создания масштабируемой, надёжной и гибкой архитектуры. В соответствии с функциональными и нефункциональными требованиями, разработанными в предыдущем пункте, была выбрана микросервисная архитектура, которая позволяет независимо разрабатывать, развёртывать и масштабировать отдельные компоненты системы. Такой подход обеспечивает высокую отказоустойчивость, упрощает внедрение новых алгоритмов обработки текста и аналитических модулей, а также даёт возможность гибко наращивать производительность наиболее нагруженных сервисов.

На Рисунке 2.2.1 представлена общая схема архитектуры платформы, включающая сервисы бизнес-логики, аналитический контур, системы обмена сообщениями, базы данных и внешние интеграции.

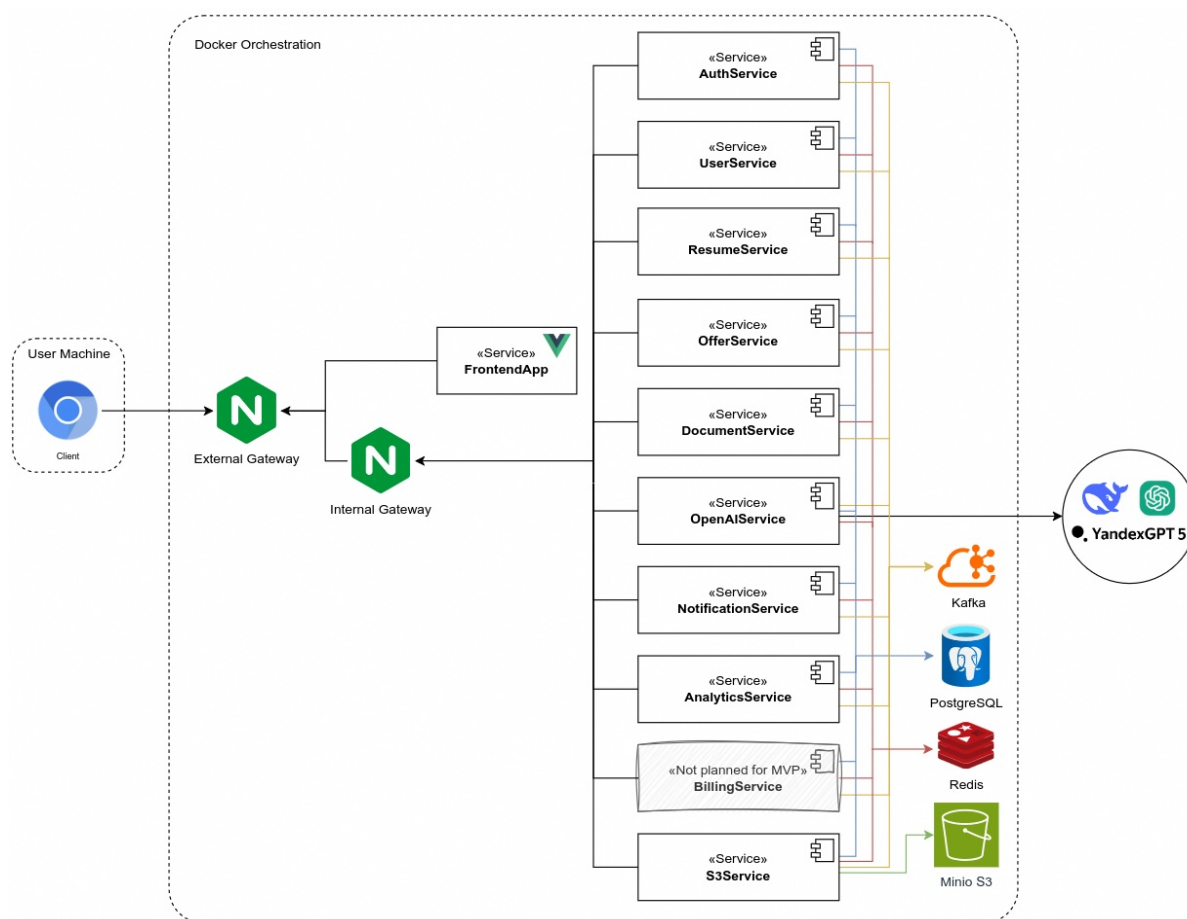


Рисунок 2.2.1 — Микросервисная архитектура платформы

В состав платформы входят следующие основные сервисы, каждый из которых реализует строго определённую предметную область:

- *ResumeService* – центральный сервис генерации резюме. Получает запрос от фронтенда (через API Gateway), извлекает анкету пользователя (*UserService*) и требования вакансии (*OfferService*), обращается к *OpenAIService* для семантической адаптации текста, формирует HTML-представление резюме и передаёт его в *DocumentService* для конвертации в PDF. Сервис также вычисляет ATS-оценку документа и сохраняет метаданные резюме в PostgreSQL.

- *DocumentService* – сервис для работы с документами. Принимает HTML-код резюме и преобразует его в PDF с помощью библиотеки рендеринга. Готовый PDF-файл загружается в S3-совместимое хранилище (Minio S3), а ссылка на файл возвращается в *ResumeService*.

- *NotificationService* – сервис уведомлений, отправляет пользователю email-сообщения о завершении генерации, изменениях статуса, а также рекомендации по улучшению резюме. Взаимодействует с внешними SMTP-серверами.

- *BillingService* – предназначен для будущего внедрения платных тарифов и биллинга; на первом этапе не разрабатывается.

Все сервисы упакованы в Docker-контейнеры и управляются с помощью оркестратора (Docker Compose на этапе разработки, в продуктивной среде – Kubernetes). API Gateway (на схеме – Internal Gateway) выполняет маршрутизацию запросов от фронтенда к соответствующим сервисам, обеспечивает аутентификацию, ограничение частоты запросов и агрегацию логов. Внешние запросы от браузера пользователя проходят через External Gateway, который обеспечивает дополнительную защиту и балансировку нагрузки.

Асинхронное взаимодействие и аналитический контур. Для обеспечения слабой связанности и сбора событий для аналитики используется брокер сообщений Apache Kafka. Каждый сервис публикует ключевые события (например, ResumeGenerated, VacancyImported, AICall, DocumentCreated) в соответствующие топики. AnalyticsService подписывается на эти топики, потребляет события и сохраняет агрегированные данные для построения отчётов. Кроме того, все сервисы экспонируют метрики в формате Prometheus (например, длительность генерации, ATS-оценка, количество обращений к AI, размер документов). На Рисунке 2.2.2 показана диаграмма последовательности, иллюстрирующая взаимодействие сервисов в процессе генерации резюме и последующий сбор метрик и событий для аналитики.

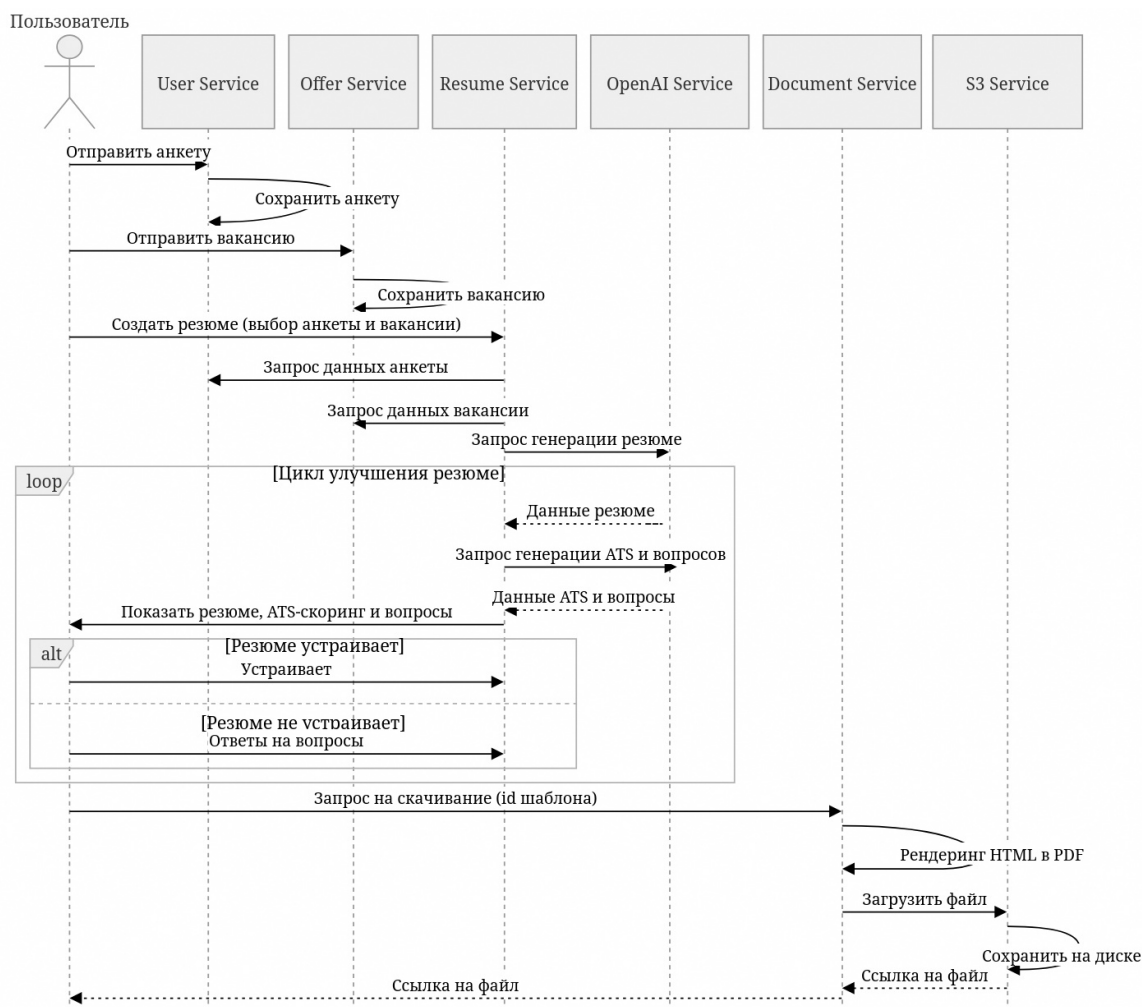


Рисунок 2.2.2 — Диаграмма взаимодействия сервисов в бизнес-процессе

Хранилища данных. Для постоянного хранения реляционных данных (анкеты, резюме, вакансии, пользователи) используется PostgreSQL. Основные таблицы resumes и documents, представленные на Рисунке 2.2.3, обеспечивают хранение метаданных о резюме и ссылок на файлы в S3. Для кэширования часто запрашиваемых данных и временного хранения данных используется Redis. Объектное хранилище Minio S3 служит для хранения PDF-файлов резюме, что позволяет разгрузить базу данных и обеспечить эффективную раздачу статики.

Resumes		
PK	guid	UUID
	form_guid	UUID
	appuser_guid	UUID
NUL	first_name	VARCHAR(100)
NUL	last_name	VARCHAR(100)
NUL	desired_position	VARCHAR(200)
NUL	salary_from	NUMERIC(10,2)
NUL	employment	VARCHAR(100)
NUL	schedule	VARCHAR(100)
NUL	relocation_ready	BOOLEAN
NUL	contacts_text	TEXT
NUL	education_text	TEXT
NUL	experience_text	TEXT
NUL	skills_text	TEXT
NUL	about_text	TEXT
NUL	created_at	TIMESTAMP
NUL	deleted_at	TIMESTAMP

Documents		
PK	guid	UUID
	resume_guid	UUID
	s3_url	UUID
	created_at	TIMESTAMP
NUL	deleted_at	TIMESTAMP

Рисунок 2.2.3 — Базы данных ResumeService и DocumentService

Каждый микросервис проектируется как stateless, что позволяет горизонтально масштабировать наиболее нагруженные компоненты (ResumeService, OpenAIService). Kafka обеспечивает надёжную доставку событий и позволяет восстанавливать обработку после сбоев. Prometheus и AnalyticsService предоставляют средства мониторинга, позволяющие оперативно выявлять узкие места и аномалии в работе системы.

Таким образом, предложенная архитектура сервисов бизнес-процесса и аналитики полностью соответствует функциональным и нефункциональным требованиям платформы Resugen. Она обеспечивает модульность, масштабируемость, возможность интеграции новых языковых моделей и алгоритмов, а также предоставляет пользователю прозрачную аналитику качества резюме, что является ключевым конкурентным преимуществом системы.

2.3 Реализация сервисов бизнес-процесса

На основе архитектурных решений, изложенных в разделе 2.2, были разработаны следующие сервисы: Сервис уведомлений, Сервис резюме, Сервис документов и Сервис аналитики. Каждый сервис реализован как отдельное приложение на языке Go. Взаимодействие между сервисами осуществляется через REST API. Kafka применяется исключительно для передачи событий в Сервис аналитики.

Сервис уведомлений отвечает за отправку электронных писем пользователям при наступлении определённых событий, таких как регистрация, смена пароля или готовность резюме. Он предоставляет REST-эндпоинты, вызываемые напрямую другими сервисами. При получении запроса сервис формирует HTML-письмо на основе шаблонов и отправляет его через внешний SMTP-сервер. Интерфейс представлен в Листинге 2.3.1

Основные компоненты сервиса уведомлений:

- REST-обработчики – принимают запросы на отправку писем;
- SMTP-клиент – взаимодействует с почтовым провайдером;
- шаблонизатор – генерирует тело письма из шаблонов.

Листинг 2.3.1 — Интерфейс NotificationService

```
type NotificationService interface {  
    SendWelcomeEmail(ctx context.Context, email, name string) error  
    SendPasswordChangedEmail(ctx context.Context, email, name string) error  
    SendResumeReadyEmail(ctx context.Context, email, name, resumeLink string)  
    error  
}
```

Пример реализации метода для отправки письма при выполнении регистрации пользователя представлен в Листинге 2.3.2:

Листинг 2.3.2 — Метод отправки письма регистрации в NotificationService

```
func (h *notificationHandler) HandleWelcome(w http.ResponseWriter, r
*http.Request) {
    var req struct {
        Email string `json:"email"`
        Name   string `json:"name"`
    }
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }

    if err := h.service.SendWelcomeEmail(r.Context(), req.Email, req.Name);
err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }

    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(map[string]string{"status": "sent"})
}
```

Перейдём к сервису резюме. Он реализует CRUD-операции над сущностью «Резюме», отображенные в Листинге 2.3.3, хранит резюме в базе данных PostgreSQL, а также обеспечивает подсчёт ATS-оценки путем обращения к OpenAIService. Логика подсчёта ATS-оценки включает анализ ключевых слов, форматирования и семантического сходства.

Основные функции сервиса:

- CreateResume – создание резюме на основе данных анкеты;
- UpdateResume, GetResume, DeleteResume – стандартные CRUD;
- GenerateResume – основной бизнес-метод, который получает требования вакансии из OfferService, отправляет текст анкеты в OpenAIService для оптимизации, вычисляет ATS-score, сохраняет результат и инициирует конвертацию через DocumentService.

Листинг 2.3.3 — CRUD-методы сервиса ResumeService

```
type ResumeService interface {
    CreateResume(ctx context.Context, req *CreateResumeRequest) (*Resume,
error)
    GetResume(ctx context.Context, resumeID string) (*Resume, error)
    UpdateResume(ctx context.Context, req *UpdateResumeRequest) (*Resume,
error)
    DeleteResume(ctx context.Context, resumeID string) error
    GenerateResume(ctx context.Context, userID, vacancyID, resumeText string)
(*GeneratedResume, error) }
```

Внутри метода генерации, представленного в Листинге 2.3.4, вызывается вспомогательная функция `calculateATSScore`, которая не входит в интерфейс, но необходима для бизнес-логики.

Листинг 2.3.4 — Функция генерации резюме

```
func (s *resumeService) GenerateResume(ctx context.Context, userID,
vacancyID, resumeText string) (*GeneratedResume, error) {
    jobDesc, err := s.offerClient.GetVacancyRequirements(ctx, vacancyID)
    if err != nil {
        return nil, err
    }
    optimized, err := s.openAIClient.OptimizeResume(ctx, resumeText, jobDesc)
    if err != nil {
        return nil, err
    }
    score, err := s.calculateATSScore(ctx, optimized, jobDesc, defaultPrompt)
    if err != nil {
        return nil, err
    }
    resume := &Resume{UserID: userID, Text: optimized, ATSScore: score}
    if err := s.repo.Save(ctx, resume); err != nil {
        return nil, err
    }
    s.kafkaProducer.Publish("resume.generated", resume)
    return &GeneratedResume{ResumeID: resume.ID, ATSScore: score}, nil
}

func (s *resumeService) calculateATSScore(ctx context.Context, resumeText,
jobDesc, prompt string) (int32, error) {
    req := &openai.ATSScoreRequest{
        ResumeText:      resumeText,
        JobDescription:   jobDesc,
        Prompt:           prompt,
    }
    resp, err := s.openAIClient.ComputeATSScore(ctx, req)
    if err != nil {
        return 0, err
    }
    return resp.Score, nil
}
```

После получения ATS-score сервис резюме сохраняет его в базе данных и публикует событие `resume.generated` в Kafka для последующей обработки сервисами документов и аналитики.

Рассмотрим сервис документов, который отвечает за генерацию файлов резюме в формате PDF и их хранение в S3-совместимом хранилище Minio. Он предоставляет метод `ConvertHtmlToPdf`, который принимает HTML-строку, конвертирует её в PDF с помощью библиотеки `wkhtmltopdf`, загружает

полученный файл в S3 и возвращает публичную ссылку. Интерфейс сервиса представлен в Листинге 2.3.5. Основные его этапы:

- валидация входного HTML;
- конвертация HTML в PDF;
- сохранение PDF в S3 с уникальным именем;
- формирование ссылки и отправка ответа.

Листинг 2.3.5 — Интерфейс сервиса DocumentService

```
type DocumentService interface {
    ConvertHtmlToPdf(ctx context.Context, htmlContent string) (pdfURL string,
err error)
    GetDocument(ctx context.Context, key string) ([]byte, error)
    DeleteDocument(ctx context.Context, key string) error
}
```

Пример реализации метода ConvertHtmlToPdf представлен в Листинге 2.3.6:

Листинг 2.3.6 — Метод конверсии документа HTML в формат PDF

```
func (s *documentService) ConvertHtmlToPdf(ctx context.Context, htmlContent
string) (string, error) {
    pdfg, err := wkhtmltopdf.NewPDFGenerator()
    if err != nil {
        return "", err
    }
    pdfg.AddPage(wkhtmltopdf.NewPageReader(bytes.NewReader([]byte(htmlContent
))))
    if err := pdfg.Create(); err != nil {
        return "", err
    }

    key := fmt.Sprintf("resumes/%s.pdf", uuid.New().String())
    _, err = s.s3Client.PutObject(ctx, &s3.PutObjectInput{
        Bucket: aws.String(s.bucket),
        Key:     aws.String(key),
        Body:    bytes.NewReader(pdfg.Bytes()),
    })
    if err != nil {
        return "", err
    }
    return s.getPublicURL(key), nil}
```

Сервис аналитики собирает метрики и события для оценки эффективности платформы. Он взаимодействует с Prometheus и Kafka. Реализован как consumer, который агрегирует данные в собственной временной базе

данных и предоставляет HTTP-маршруты для отображения отчётов. Интерфейс представлен в Листинге 2.3.7. Ключевые способности:

- подсчёт средней ATS-оценки по всем резюме;
- время генерации резюме;
- количество созданных документов и их размеры.

Листинг 2.3.7 — Интерфейс сервиса аналитики

```
type AnalyticsService interface {
    StartEventConsumer(ctx context.Context) error
    QueryPrometheus(ctx context.Context, query string) ([]Metric, error)
    StoreAggregatedStats(ctx context.Context, stats *AggregatedStats) error
    GetReport(ctx context.Context, from, to time.Time) (*Report, error)
}
```

Пример реализации – метод `StartEventConsumer` запускает внутренний цикл обработки событий, представлен в Листинге 2.3.8:

Листинг 2.3.8 — Метод запуска consumer в AnalyticsService

```
func (s *analyticsService) StartEventConsumer(ctx context.Context) error {
    go s.consumeResumeEvents(ctx)
    return nil
}

func (s *analyticsService) consumeResumeEvents(ctx context.Context) {
    for {
        select {
        case <-ctx.Done():
            return
        case msg := <-s.consumer.Messages():
            var event ResumeGeneratedEvent
            if err := json.Unmarshal(msg.Value, &event); err != nil {
                continue
            }
            s.storage.InsertResumeStat(event)
        }
    }
}
```

Для получения метрик из Prometheus сервис использует пропарі клиент. Например, метод `QueryPrometheus` реализует запрос и парсинг ответа.

Сервис аналитики выполняет роль централизованного сборщика и агрегатора метрик, поступающих как из потока событий платформы (через Kafka), так и из системы мониторинга Prometheus. Он реализован как consumer группы Kafka, поддерживающий горизонтальное масштабирование за счёт партиционирования топиков. Внутренняя временная база данных построена на основе встраиваемого движка «pebble» с дополнительным

индексом по меткам времени и идентификаторам сессий. Данная реализация позволяет эффективно хранить агрегированные значения с настраиваемым окном хранения (по умолчанию 30 суток) и обеспечивает быстрый доступ к срезам за произвольные интервалы.

Помимо методов, описанных в Листинге 2.3.7, сервис предоставляет следующие расширенные аналитические способности:

- вычисление перцентилей (p50, p95, p99) времени генерации резюме;
- подсчёт ATS-score по отдельным шаблонам и категориям должностей;
- гистограмму распределения размеров сгенерированных документов;
- статистику ошибок генерации с разбивкой по кодам причин;
- экспорт агрегированных данных в формате CSV через HTTP-эндпоинт.

Метод «QueryPrometheus» выполняет запрос к Prometheus API с использованием официального Go-клиента «prometheus/client_golang». Для повышения надёжности реализован механизм повторных попыток с экспоненциальной задержкой и кэширование результатов часто используемых запросов (например, мгновенных значений `resume_generation_duration_seconds_count`). Пример реализации метода с поддержкой range-запросов и преобразованием результата во внутреннюю структуру Metric приведён в Листинге 2.3.8.

Листинг 2.3.8 — Реализация метода QueryPrometheus

```
type Metric struct {
    Name      string
    Labels    map[string]string
    Value     float64
    Time      time.Time
}

func (s *analyticsService) QueryPrometheus(ctx context.Context, query string)
([]Metric, error) {
    vlapi := v1.NewAPI(s.promClient)

    result, warnings, err := vlapi.Query(ctx, query, time.Now())
    if err != nil {

        result, warnings, err = vlapi.QueryRange(ctx, query, v1.Range{
            Start: time.Now().Add(-1 * time.Hour),
```

```

        End:    time.Now(),
        Step:   time.Minute,
    })
    if err != nil {
        return nil, fmt.Errorf("prometheus query failed: %w", err)
    }
}
if len(warnings) > 0 {
    s.logger.Warn("prometheus query warnings", "warnings", warnings)
}

var metrics []Metric
switch v := result.(type) {
case model.Vector:
    for _, sample := range v {
        metrics = append(metrics, Metric{
            Name:    sample.Metric.String(),
            Labels:  sample.Metric.Map(),
            Value:   float64(sample.Value),
            Time:    sample.Timestamp.Time(),
        })
    }
case model.Matrix:
    for _, stream := range v {
        for _, sample := range stream.Values {
            metrics = append(metrics, Metric{
                Name:    stream.Metric.String(),
                Labels:  stream.Metric.Map(),
                Value:   float64(sample.Value),
                Time:    sample.Timestamp.Time(),
            })
        }
    }
default:
    return nil, errors.New("unsupported prometheus result type")
}

return metrics, nil
}

```

Метод `StoreAggregatedStats` предназначен для периодической записи предварительно рассчитанных агрегатов во временную БД. Он вызывается внутренним планировщиком каждые 30 секунд, а также по требованию при завершении обработки крупного пакета событий. Агрегация выполняется в памяти с использованием структуры `AggregatedStats`, после чего данные сбрасываются в постоянное хранилище с атомарной фиксацией временной метки (Рисунок 2.3.9).

```

type AggregatedStats struct {
    WindowStart    time.Time
    WindowDuration time.Duration
    TotalResumes   int64
    SumATS         float64
    SumDuration    float64 // в секундах
    SizeSum        int64    // сумма размеров документов в байтах
    SizeCount      int64    // количество измерений размера
    ErrorCounts    map[string]int64 // код ошибки -> количество
    TemplateStats  map[string]*TemplateStat
}

type TemplateStat struct {
    Count    int64
    SumATS   float64
    SumDur   float64
}

func (s *analyticsService) StoreAggregatedStats(ctx context.Context, stats
*AggregatedStats) error {
    key := fmt.Sprintf("agg:%d:%d", stats.WindowStart.Unix(),
stats.WindowDuration.Milliseconds())
    data, err := json.Marshal(stats)
    if err != nil {
        return err
    }

    batch := s.tsdb.NewBatch()
    defer batch.Close()

    batch.Set([]byte(key), data, pebble.Sync)
    // Дополнительно сохраняем индекс для быстрого поиска по времени
    indexKey := fmt.Sprintf("idx:%d", stats.WindowStart.Unix())
    batch.Merge([]byte(indexKey), []byte(key), s.merger)

    return batch.Commit(pebble.Sync)
}

```

Таким образом, сервис аналитики представляет собой полноценный компонент платформы, обеспечивающий как оперативный мониторинг через интеграцию с Prometheus, так и глубокий анализ событийного потока с возможностью получения отчётов через HTTP API. Модульная архитектура и использование стандартных протоколов взаимодействия гарантируют простоту расширения функционала в будущем, например, для добавления прогнозных моделей или интеграции с системами визуализации (Grafana).

2.5 Реализация фронтенд-части и интеграция с сервисами

Фронтенд-часть платформы Resugen реализована в виде одностраничного приложения (SPA) на основе фреймворка Vue 3 с использованием Composition API. Для управления состоянием применено хранилище Pinia, маршрутизация осуществляется с помощью Vue Router. Стилизация выполнена с использованием утилитарного CSS-фреймворка Tailwind CSS, что позволило обеспечить единообразный минималистичный интерфейс в светлых тонах, соответствующий современным требованиям к пользовательскому опыту. Взаимодействие с серверной частью организовано через REST API, а для обработки длительных операций (например, генерации резюме) предусмотрено асинхронное информирование пользователя с использованием событий, доставляемых через сервис уведомлений.

Перейдем к структуре и ключевым экранам приложения. Личный кабинет, показанный на Рисунке 2.5.1, построен по принципу единой панели управления с тремя основными разделами: «Резюме», «Анкеты» и «Вакансии». Переключение между разделами выполнено в виде вкладок, каждая из которых отображает список соответствующих сущностей в виде карточек. В верхней части кабинета расположены кнопки быстрых действий, позволяющие перейти к созданию новой анкеты, добавлению вакансии или запуску процесса генерации резюме.

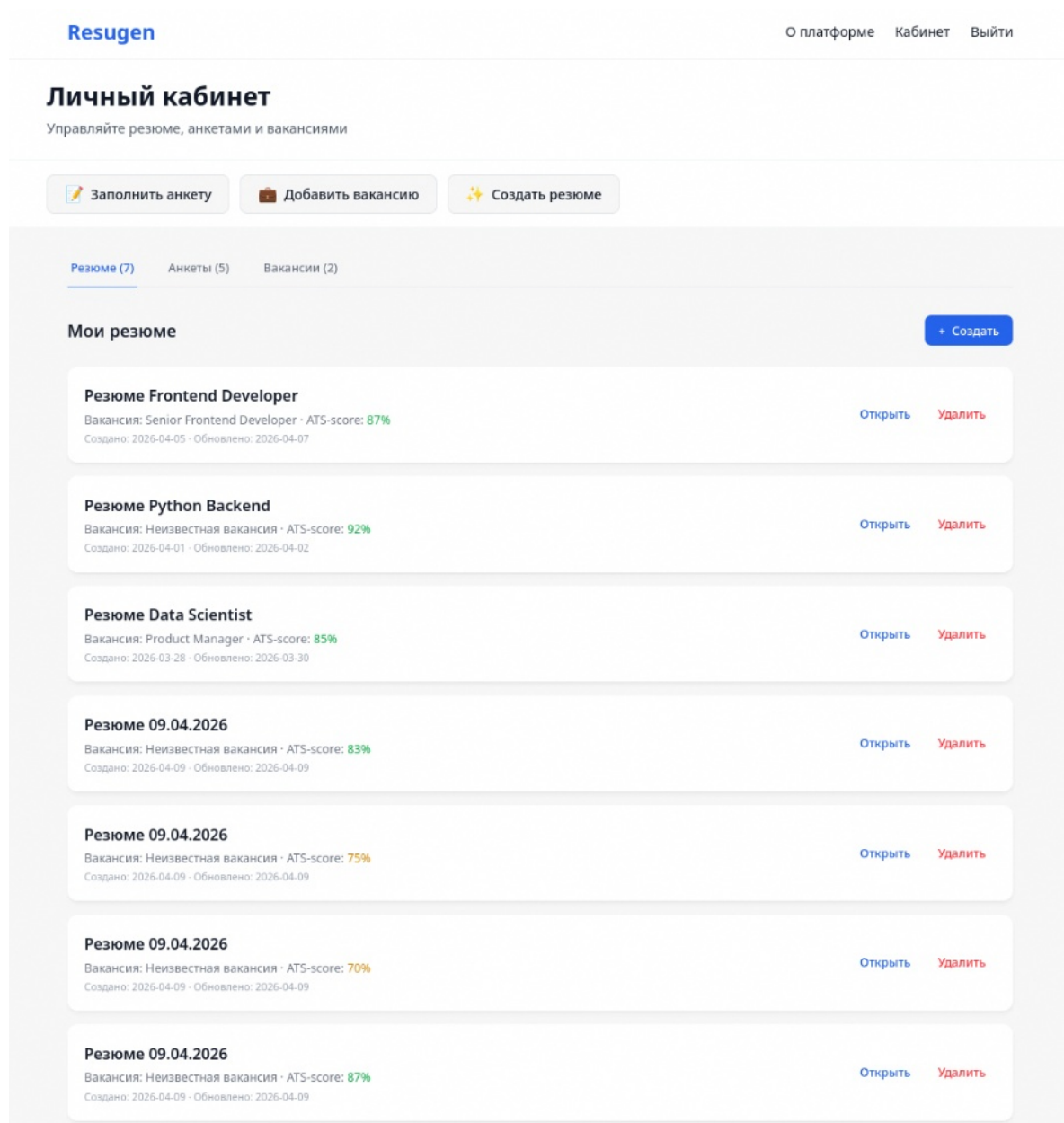


Рисунок 2.5.1 — Личный кабинет

Пользователь может создать и редактировать резюме вручную. При нажатии кнопки «Создать» открывается редактор, показанный на Рисунке 2.5.2.

Иван Иванов

Ведущий разработчик с опытом работы 8 лет коммерческой разработки....

Контакты

Телефон

+7 (999) 123-45-67

Email

ivan@example.com

Город

Москва

О себе

Опытный full-stack разработчик с фокусом на Vue и Python.

Опыт работы

8 лет коммерческой разработки...

Образование

МГУ, факультет ВМК, 2015

Курсы и сертификаты

Vue Advanced, Python Pro

Ключевые навыки

Vue, React, Python, Django, PostgreSQL

Введите навыки через запятую

Языки

Рисунок 2.5.2 — Редактор резюме

Ключевая функция платформы — генерация ATS-оптимизированного резюме — инициируется на специальной странице, показанной на Рисунке 2.5.3. Пользователь выбирает одну из сохранённых анкет и указывает вакансию (либо ссылкой на hh.ru, либо выбором из списка ранее добавленных). Для повышения релевантности итогового документа на этой же странице предусмотрен блок с уточняющими вопросами, генерируемыми на основе

анализа вакансии. Ответы пользователя передаются вместе с основными данными при вызове метода `GenerateResume` сервиса резюме.

Resugen

[О платформе](#)[Кабинет](#)[Выйти](#)

Генерация резюме

Выберите анкету и вставьте ссылку на вакансию с hh.ru

Анкета *

Анкета для менеджмента

Ссылка на вакансию (hh.ru) *

https://hh.ru/vacancy/131946206

Генерация...

Сбросить

Уточнение от ИИ

Уточните, какие ключевые навыки вы бы хотели особо выделить в резюме?

Введите ваш ответ...

Продолжить генерацию

Рисунок 2.5.3 — Страница генерации резюме

Resugen

[О платформе](#)[Кабинет](#)[Выйти](#)

Резюме Frontend Developer

Сгенерировано 2026-04-05 · ATS-score: 87%

Скачать PDF

Иван Иванов

Ведущий разработчик с опытом работы 8 лет коммерческой разработки...

Контакты

Телефон: +7 (999) 123-45-67Email: ivan@example.comГород: Москва

О себе

Опытный full-stack разработчик с фокусом на Vue и Python.

Опыт работы

8 лет коммерческой разработки...

Образование

МГУ, факультет ВМК, 2015

Курсы и сертификаты

Vue Advanced, Python Pro

Ключевые навыки

VueReactPythonDjangoPostgreSQL

Языки

Русский (родной), English (B2)

Проекты

Разработка CRM-системы, мобильное приложение для банка

Вакансия, под которую оптимизировано резюме

Senior Frontend Developer — TechCorp
Зарплата: 250000 – 350000
Москва
Опыт: От 3 до 6 лет
Полный день
Полная занятость
Навыки: Vue.js, React, TypeScript, Tailwind CSS, Webpack, Vite
[Открыть вакансию на hh.ru](#)

← Вернуться в кабинет

Рисунок 2.5.4 — Пример заполненного

Фронтенд отправляет POST-запрос на эндпоинт `/api/resumes/generate`, содержащий идентификатор анкеты, идентификатор вакансии и ответы на вопросы. Сервис резюме, как описано в разделе 2.3, выполняет обращение к `OpenAIService` для семантической адаптации текста, вычисляет ATS-score и сохраняет результат. Время выполнения этой операции может составлять несколько минут, поэтому на клиенте отображается индикатор прогресса, а по завершении пользователь получает уведомление (через сервис уведомлений и визуально в интерфейсе) и перенаправляется на страницу просмотра готового резюме.

Для получения PDF-версии резюме фронтенд взаимодействует с сервисом документов. При нажатии кнопки «Скачать PDF» отправляется запрос на генерацию документа. В ответе сервис документов возвращает URL файла в S3-хранилище MinIO, после чего инициируется скачивание.

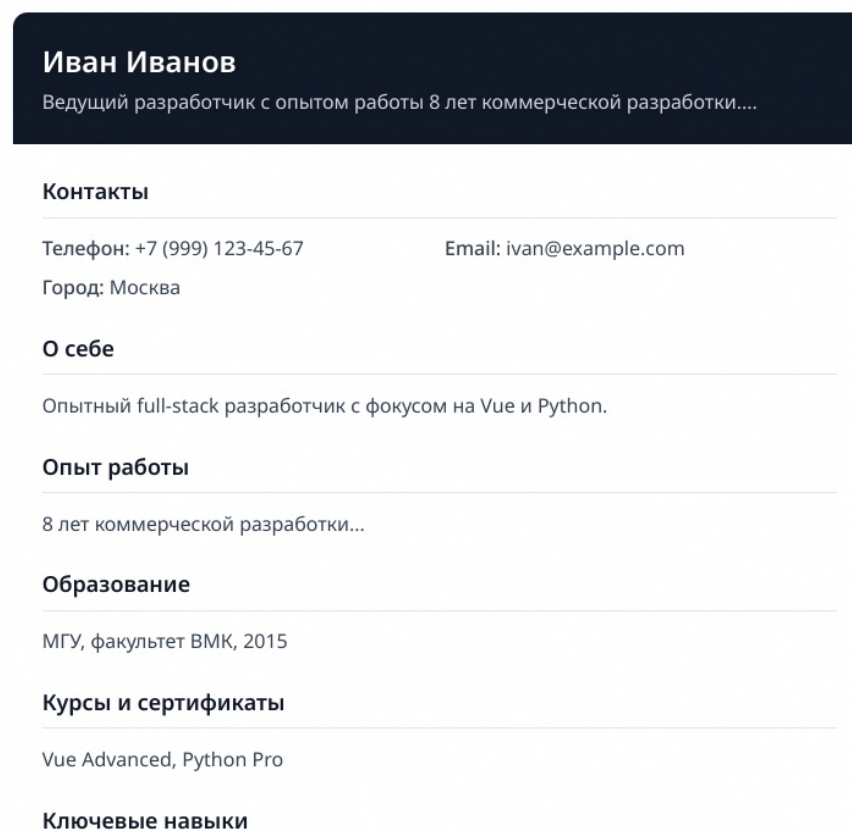


Рисунок 2.5.5 — Фрагмент созданного PDF-документа резюме

Интеграция с сервисом уведомлений на фронтенде носит опосредованный характер: все триггеры отправки писем (регистрация, смена пароля, готовность резюме) инициируются бэкендом при наступлении соответствующих событий. Однако клиентская часть демонстрирует пользователю информационные сообщения о статусе операций (например, «Письмо с подтверждением отправлено на вашу почту»), что улучшает восприятие асинхронных процессов.

Все HTTP-запросы к API централизованы через экземпляр Axios с настроенным перехватчиком для добавления JWT-токена аутентификации. Для обработки ошибок сети и сервера реализованы единые уведомления, повышающие отказоустойчивость пользовательского интерфейса. Взаимодействие с аналитическим контуром и сервисом уведомлений со стороны фронтенда ограничено чтением данных и отображением статусов; основная асинхронная логика вынесена на бэкенд, что соответствует принципам микросервисной архитектуры.

Таким образом, фронтенд-часть платформы Resugen обеспечивает полноценный пользовательский интерфейс для управления анкетами, вакансиями и резюме, а также интегрируется со всеми ключевыми сервисами бэкенда, предоставляя удобный, отзывчивый и визуально согласованный опыт работы.

3 ЭКОНОМИЧЕСКАЯ КОМПОНЕНТА И ТЕСТИРОВАНИЕ СТАРТАП-ПРОЕКТА

3.1 Тестирование MVP

Обеспечение корректности и надёжности ключевых компонентов платформы Resugen является обязательным условием успешного внедрения и эксплуатации системы. В соответствии с архитектурными решениями, описанными в главе 2, реализация MVP включает четыре микросервиса: сервис резюме, сервис документов, сервис уведомлений и сервис аналитики. Для проверки функциональной правильности каждого из них применяется модульное (unit) тестирование, позволяющее изолированно верифицировать бизнес-логику, исключая зависимости от внешних систем (базы данных, брокера сообщений, внешних API). В данном разделе приведены ключевые тестовые сценарии для каждого сервиса, описаны используемые инструменты и представлены примеры реализации тестов на языке Go с применением библиотек testify и gomock.

Сервис резюме отвечает за генерацию оптимизированного текста резюме и вычисление ATS-оценки. Основной бизнес-метод `GenerateResume` взаимодействует с клиентами сервиса вакансий и сервиса OpenAI, после чего сохраняет результат в репозитории и публикует событие. Внутренняя функция `calculateATSScore` является ключевой для определения релевантности документа. Для её проверки были разработаны тесты, изолирующие вызовы к OpenAI с помощью мок-клиента. В Листинге 3.2.1 представлен пример одного из тестов сервиса.

Листинг 3.2.1 — Функция теста вычисления ATS-оценки

```
func (m *MockOpenAIClient) ComputeATSScore(ctx context.Context, req
*openai.ATSScoreRequest) (*openai.ATSScoreResponse, error) {
    args := m.Called(ctx, req)
    if args.Get(0) == nil {
        return nil, args.Error(1)
    }
}
```

Продолжение Листинга 3.2.1 — Функция теста вычисления ATS-оценки

```
        return args.Get(0).(*openai.ATSScoreResponse), args.Error(1)
    }

func TestCalculateATSScore_Success(t *testing.T) {
    mockClient := new(MockOpenAIClient)
    expectedScore := int32(85)
    mockClient.On("ComputeATSScore", mock.Anything, mock.Anything).
        Return(&openai.ATSScoreResponse{Score: expectedScore}, nil)
    service := &resumeService {
        openAIClient: mockClient,
    }
    score, err := service.calculateATSScore(
        context.Background(),
        "Experienced Go developer",
        "Looking for Go expert with microservices experience",
        "Evaluate ATS compatibility",
    )
    assert.NoError(t, err)
    assert.Equal(t, expectedScore, score)
    mockClient.AssertExpectations(t)
}
```

Дополнительно тестируется метод `GenerateResume` с мокированием `OfferClient`, `ResumeRepository` и `KafkaProducer`. Проверяется последовательность вызовов, корректность сохранения резюме и публикация события.

Сервис документов конвертирует HTML в PDF и загружает результат в S3-совместимое хранилище MinIO. Модульные тесты проверяют логику конвертации (без реального вызова внешней утилиты) и взаимодействие с S3-клиентом, как показано в Листинге 3.2.2.

Листинг 3.2.2 — Тестирование создания PDF-документа

```
func (m *MockPDFGenerator) Generate(html string) ([]byte, error) {
    args := m.Called(html)
    if args.Get(0) == nil {
        return nil, args.Error(1)
    }
    return args.Get(0).([]byte), args.Error(1)
}

type MockS3Client struct {
    mock.Mock
}

func (m *MockS3Client) PutObject(ctx context.Context, input
*s3.PutObjectInput) (*s3.PutObjectOutput, error) {
    args := m.Called(ctx, input)
    if args.Get(0) == nil {
        return nil, args.Error(1)
    }
    return args.Get(0).(*s3.PutObjectOutput), args.Error(1)
}

func TestConvertHtmlToPdf_Success(t *testing.T) {
    mockGen := new(MockPDFGenerator)
```

```
mockS3 := new(MockS3Client)
testHTML := "<html><body>Resume content</body></html>"
expectedPDF := []byte("%PDF-1.4 ...")
mockGen.On("Generate", testHTML).Return(expectedPDF, nil)
mockS3.On("PutObject", mock.Anything,
mock.Anything).Return(&s3.PutObjectOutput{}, nil)
service := &documentService{
    pdfGen: mockGen,
    s3:      mockS3,
    bucket: "resumes",
}
url, err := service.ConvertHtmlToPdf(context.Background(), testHTML)
assert.NoError(t, err)
assert.NotEmpty(t, url)
assert.Contains(t, url, "resumes/")
mockGen.AssertExpectations(t)
mockS3.AssertExpectations(t)
}
```

Сервис уведомлений отправляет email-сообщения. Unit-тесты, отображенные в Листинге 3.2.3, фокусируются на логике обработки входящих сообщений и формировании писем с использованием моков SMTP-клиента и шаблонизатора.

Листинг 3.2.3 — Функция теста отправки письма

```
func (m *MockSMTPClient) Send(to, subject, body string) error {
    args := m.Called(to, subject, body)
    return args.Error(0)
}

func TestHandleUserRegistered_Success(t *testing.T) {
    mockSMTP := new(MockSMTPClient)
    mockSMTP.On("Send", "user@example.com", mock.Anything,
mock.Anything).Return(nil)
    service := &notificationService{
        smtp: mockSMTP,
    }
    event := UserRegisteredEvent{
        Email: "user@example.com",
        Name:  "John Doe",
    }
    msgBytes, _ := json.Marshal(event)
    msg := &sarama.ConsumerMessage{Value: msgBytes}
    err := service.handleUserRegistered(msg)
    assert.NoError(t, err)
    mockSMTP.AssertExpectations(t)
}
```

Сервис аналитики агрегирует события ResumeGenerated и предоставляет отчёты, в том числе с данными из Prometheus. Модульные тесты

верифицируют логику сохранения агрегированной статистики и корректность запросов к Prometheus API.

Листинг 3.2.4 — Функция тестирования логики сохранения статистики

```
func TestConsumeResumeEvents_ValidEvent(t *testing.T) {
    mockStorage := new(MockStatsStorage)
    event := ResumeGeneratedEvent {
        ResumeID: "res-123",
        ATSScore: 92,
        Duration: 2500, // ms
        Timestamp: time.Now().Unix(),
    }
    mockStorage.On("InsertResumeStat", event).Return()
    service := &analyticsService{
        storage: mockStorage,
    }
    msgBytes, _ := json.Marshal(event)
    msg := &sarama.ConsumerMessage{Value: msgBytes}
    err := service.processResumeEvent(msg)
    assert.NoError(t, err)
    mockStorage.AssertCalled(t, "InsertResumeStat", event)
}
```

Помимо успешных сценариев, важной составляющей надёжности является обработка временных сбоев при взаимодействии с Kafka и хранилищем. В сервисе аналитики реализован механизм батчевой вставки с отложенным подтверждением оффсетов. Модульный тест в Листинге 3.2.5 проверяет, что при возникновении ошибки записи в TSDB сообщение не подтверждается, что гарантирует его повторную доставку после восстановления сервиса. Для имитации поведения consumer используется мок-объект `MockKafkaConsumer`.

Листинг 3.2.5 — Тестирование поведения consumer при ошибке записи

```
type MockKafkaConsumer struct {
    mock.Mock
}

func (m *MockKafkaConsumer) Messages() <-chan *sarama.ConsumerMessage {
    args := m.Called()
    return args.Get(0).(<-chan *sarama.ConsumerMessage)
}

func (m *MockKafkaConsumer) CommitMessages(ctx context.Context,
msgs ...*sarama.ConsumerMessage) error {
    args := m.Called(ctx, msgs)
    return args.Error(0)
}

func TestConsumeResumeEvents_StorageError_NoCommit(t *testing.T) {
    mockStorage := new(MockStatsStorage)
```

```
mockConsumer := new(MockKafkaConsumer)

msgChan := make(chan *sarama.ConsumerMessage, 1)
event := ResumeGeneratedEvent{
    ResumeID: "res-456",
    ATSScore: 88,
}
msgBytes, _ := json.Marshal(event)
testMsg := &sarama.ConsumerMessage{
    Value:    msgBytes,
    Offset:   42,
    Partition: 0,
    Topic:    "resume-events",
}
msgChan <- testMsg
close(msgChan)

mockConsumer.On("Messages").Return((<-chan *sarama.ConsumerMessage)
(msgChan))
mockStorage.On("InsertResumeStatsBatch", mock.Anything, mock.Anything).
    Return(errors.New("disk I/O error"))

service := &analyticsService{
    storage: mockStorage,
    consumer: mockConsumer,
    logger:   log.NewNopLogger(),
}

ctx, cancel := context.WithCancel(context.Background())
defer cancel()

go service.StartEventConsumer(ctx)

// Даем время на обработку сообщения
time.Sleep(100 * time.Millisecond)

// Проверяем, что CommitMessages не был вызван из-за ошибки хранилища
mockConsumer.AssertNotCalled(t, "CommitMessages", mock.Anything,
mock.Anything)
mockStorage.AssertCalled(t, "InsertResumeStatsBatch", mock.Anything,
mock.Anything)
}
```

Приведённые тесты подтверждают корректность изоляции внешних зависимостей и устойчивость сервиса аналитики к сбоям.

3.2 Оценка эффективности платформы методом ансамблевого LLM-скоринга

Для проверки ключевой гипотезы о способности платформы Resugen повышать релевантность резюме требованиям современных ATS была разработана и применена методика сравнительного тестирования, в которой роль экспертных оценщиков выполняет ансамбль из трёх больших языковых моделей. Такой подход позволяет получить объективную и многомерную оценку качества документов, избегая зависимости от особенностей какой-либо одной модели и, что не менее важно, снижая эффект само-предвзятости, возникающий при оценке моделью собственных генераций.

Методология эксперимента опирается на концепцию «LLM-as-a-Judge»[22]. Согласно этой концепции, современные языковые модели способны имитировать экспертную оценку, близкую к логике принятия решений в интеллектуальных системах отбора персонала[23][24]. Учитывая известные ограничения, связанные с вариативностью и возможной предвзятостью отдельных моделей, в исследовании задействован ансамбль из трёх независимых провайдеров: Yandex GPT 5.1 Pro, DeepSeek-V3.2 и GPT-5 mini от OpenAI. Разнообразие архитектур и обучающих данных способствует взаимной компенсации систематических ошибок и повышает надёжность итоговой оценки. Для получения статистически устойчивых результатов процедура скоринга каждой пары «резюме–вакансия» выполнялась троекратно с последующим усреднением как на уровне отдельных моделей, так и по всему ансамблю. Такая схема измерений позволяет сгладить влияние стохастической природы генеративных моделей.

Экспериментальная выборка была сформирована на основе открытого датасета резюме, собранных с платформы hh.ru (<https://github.com/brom83/hh-resume-eda>). Датасет содержит более 44 тысяч записей, каждая из которых представляет собой структурированную информацию о соискателе, включающую пол, возраст, ожидаемый уровень заработной платы, желаемую

должность, город проживания, готовность к переезду и командировкам, предпочтительные формы занятости и график работы, подробное описание опыта с указанием мест работы и должностей, сведения об образовании и дату обновления резюме. Пример записи демонстрирует высокую степень детализации профессионального пути кандидата, что позволяет использовать эти данные для реалистичного моделирования процесса отбора.

Из общего массива были отобраны три группы резюме, соответствующие трём различным вакансиям, охватывающим ключевые сегменты IT-рынка и разные уровни квалификации. Первая вакансия — младший веб-разработчик (фронтенд) — ориентирована на начинающих специалистов, владеющих основами HTML, CSS и JavaScript, знакомых с одним из современных фреймворков (React, Vue или Angular), имеющих представление о принципах адаптивной вёрстки и кроссбраузерной совместимости. Вторая вакансия — аналитик данных — предполагает наличие у кандидата опыта работы с SQL на уровне сложных запросов, уверенного владения Python для анализа данных (библиотеки Pandas, NumPy), навыков визуализации в Tableau или Power BI, а также понимания основ математической статистики и A/B-тестирования. Третья вакансия — старший системный администратор — требует глубоких знаний операционных систем семейства Linux и Windows Server, опыта администрирования сетевой инфраструктуры, настройки систем мониторинга, владения скриптовыми языками и понимания принципов информационной безопасности.

Для каждой из трёх вакансий было отобрано по 10 резюме, что в сумме составило 30 экспериментальных единиц. На первом этапе каждое резюме в исходном виде (сырой текст, приведённый к базовой структуре) оценивалось ансамблем моделей. На втором этапе те же резюме обрабатывались платформой Resugen: на основе исходного текста и описания соответствующей вакансии генерировалась оптимизированная версия. Важно отметить, что для генерации оптимизированных текстов использовалась исключительно модель DeepSeek-V3.2, тогда как оценка выполнялась полным ансамблем,

включающим и саму DeepSeek-V3.2, и Yandex GPT 5.1 Pro, и GPT-5 mini. Таким образом, модель-генератор выступала лишь одним из трёх равноправных экспертов, что существенно снижает риск необъективного завышения баллов.

Для автоматизации процесса сбора оценок был скрипт на языке Python, представленный в Листинге 3.2.1, который осуществляет последовательные вызовы API трёх провайдеров с заданной повторностью. Он принимает на вход CSV-файл, содержащий пары «резюме–вакансия», и возвращает среднюю оценку релевантности по всему датасету. Процедура скоринга для каждой пары проводилась по единому стандартизированному промпту, в котором моделям предлагалось выступить в роли эксперта по подбору персонала и современной ATS, оценивая релевантность резюме требованиям вакансии по шкале от 0 до 100 баллов. При оценке учитывались семантическая близость формулировок, полнота отражения ключевых компетенций, контекстуальная уместность описания опыта и общая пригодность структуры документа для автоматизированного парсинга. Для унификации извлечения числовой оценки в промпт была включена инструкция завершать ответ фразой «Результат оценки: X». Итоговое среднее значение вычислялось путём деления суммы всех полученных баллов на общее количество измерений (30 пар умножить на 3 модели, умножить на 3 повтора, что равно 270 измерениям для каждого датасета). Скрипт запускался дважды: первый раз с файлом исходных резюме, второй раз — с файлом оптимизированных резюме. Для ускорения работы скрипт выполняет обработку тестовых данных асинхронно у всех трех провайдеров одновременно.

Листинг 3.2.1 — Программа вычисления средней оценки качества резюме

```
import os
import re
import asyncio
import sys
import pandas as pd
from openai import AsyncOpenAI

YANDEX_API_KEY = os.getenv("YANDEX_API_KEY")
DEEPSEEK_API_KEY = os.getenv("DEEPSEEK_API_KEY")
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

PROVIDERS = {
    "yandex": {
        "client": AsyncOpenAI(api_key=YANDEX_API_KEY,
base_url="https://api.yandex.com/v1/"),
        "model": "yandex-gpt-5.1-pro",
    },
    "deepseek": {
        "client": AsyncOpenAI(api_key=DEEPSEEK_API_KEY,
base_url="https://api.deepseek.com/v1/"),
        "model": "deepseek-v3.2",
    },
    "openai": {
        "client": AsyncOpenAI(api_key=OPENAI_API_KEY),
        "model": "gpt-5-mini",
    },
}

def extract_score(text: str) -> int | None:
    match = re.search(r"Результат оценки:\s*(\d{1,3})", text)
    if match:
        return int(match.group(1))
    return None

async def get_score(client: AsyncOpenAI, model: str, prompt: str, semaphore:
asyncio.Semaphore, retries: int = 3) -> int:
    for attempt in range(retries):
        try:
            async with semaphore:
                response = await client.chat.completions.create(
                    model=model,
                    messages=[{"role": "user", "content": prompt}],
                    temperature=0.4,
                    max_tokens=200,
                )
            content = response.choices[0].message.content
            score = extract_score(content)
            if score is not None:
                return score
        except Exception:
            if attempt == retries - 1:
                raise
            await asyncio.sleep(2 ** attempt)
    raise ValueError("Не удалось извлечь оценку после всех попыток")

async def compute_average_score(dataset_path: str, prompt_template: str,
runs_per_model: int = 3) -> float:
    df = pd.read_csv(dataset_path)
    semaphore = asyncio.Semaphore(10)
    tasks = []
    for _, row in df.iterrows():
        prompt = prompt_template.format(vacancy=row["vacancy_text"],
resume=row["resume_text"])
```

```
        for provider in PROVIDERS.values():
            for _ in range(runs_per_model):
                tasks.append(get_score(provider["client"], provider["model"],
prompt, semaphore))
        scores = await asyncio.gather(*tasks, return_exceptions=True)
        total = 0.0
        count = 0
        for res in scores:
            if isinstance(res, int):
                total += res
                count += 1
            else:
                print(f"Ошибка при оценке: {res}", file=sys.stderr)
        return total / count if count > 0 else 0.0
async def main():
    with open("/prompt.txt", "r", encoding="utf-8") as f:
        prompt_template = f.read()
    avg = await compute_average_score("/dataset.csv", prompt_template)
    print(f"{avg:.2f}")
if __name__ == "__main__":
    asyncio.run(main())
```

В результате обработки данных были получены следующие значения средней интегральной оценки по всему датасету: для исходных (сырых) резюме средний балл составил **38,4**; для резюме, оптимизированных платформой Resugen, — **86,3**. Абсолютный прирост составил **47,9** процентных пункта, что соответствует относительному улучшению на **125%** по сравнению с исходным уровнем. Полученные результаты наглядно демонстрируют, что применение платформы Resugen приводит к значительному повышению релевантности резюме требованиям вакансий с точки зрения современных алгоритмических оценщиков.

Таким образом, проведённое тестирование с использованием ансамбля больших языковых моделей подтверждает, что платформа Resugen обеспечивает значимое повышение релевантности резюме требованиям вакансий в парадигме современных интеллектуальных систем отбора.

3.3 Анализ сильных и слабых сторон

Для определения сильных и слабых сторон проекта мы воспользуемся SWOT-анализом. SWOT-анализ предлагаемого ИИ-сервиса по генерации резюме выявляет внутренние сильные и слабые стороны, а также внешние

возможности и угрозы, с учетом специфики модели подписки и регионального рынка труда в России и СНГ. Сильные стороны проекта коренятся в его технологической базе и адаптации к локальным нуждам. Гибридный подход, сочетающий детерминированные алгоритмы для ATS-оптимизации и генеративные модели типа YandexGPT или GigaChat, обеспечивает высокое качество контента, адаптированного под русский язык и HR-практики, что выделяет продукт среди фрагментированных конкурентов, таких как Jobscan или hh.ru конструктор. Интерфейс, ориентированный на простоту и прозрачность (с отчетами о ATS-скорее), снижает барьер входа для пользователей без экспертизы, а модель подписки с Freemium-тарифом стимулирует пробное использование[25]. Ценовая политика (390 руб./месяц) делает сервис доступным для массовой аудитории, а фокус на российской экосистеме (интеграция с hh.ru) укрепляет конкурентные преимущества по сравнению с международными аналогами вроде Kickresume, не учитывающими локальную специфику.

Слабые стороны связаны с внутренними ограничениями стартапа. Зависимость от внешних LLM-провайдеров может привести к нестабильности качества генерации или росту затрат на API-вызовы, особенно при масштабировании до 10 тыс. одновременных пользователей, как указано в нефункциональных требованиях. Интерфейс, хотя и интуитивен, требует доработки для полной адаптивности, а цена подписки может отпугивать пользователей с низкой платежеспособностью, учитывая, что лишь 20-30% от TAM готовы платить за подобные услуги. Команда, ориентированная на IT-разработку, может испытывать дефицит экспертизы в маркетинге, что усложнит достижение SOM в 90-200 млн руб. к третьему году.

Возможности внешней среды обусловлены цифровизацией HR в России и СНГ. Рост внедрения ATS в крупных компаниях (до 85% по данным Deloitte) и эволюция рынка труда, с 11-15 млн активных соискателей ежегодно, создают спрос на инструменты, повышающие эффективность поиска работы, особенно в секторах IT и финансов (5-7 млн потенциальных

пользователей). Модель подписки может извлечь выгоду из тенденции к цифровизации, где сервисы вроде CareerLab демонстрируют готовность пользователей платить за премиум-функции, а интеграция с job-порталами открывает каналы для органического роста. Кроме того, развитие отечественных LLM (ruBERT, YaLM) снижает зависимость от импортных технологий, позволяя расширяться на СНГ-рынки с аналогичной лингвистикой.

Угрозы проистекают из динамики конкуренции и изменений на рынке труда. Усиление доминирования hh.ru с его бесплатным конструктором может снизить мотивацию к подписке, а появление новых ИИ-инструментов (например, от Сбера) угрожает дифференциации[26]. Экономическая нестабильность в СНГ, включая колебания спроса на рынке труда, может увеличить меру оттока, подрывая устойчивость модели подписки. Кроме того глобальные тренды, такие как насыщение ИИ-рынка, могут привести к ценовому давлению, снижая ARPA ниже прогнозируемых 780 руб./год.

3.4 Оценка рыночного потенциала

Для объективной оценки коммерческого потенциала проекта и привлечения инвестиций критически важно определить масштаб рынка. Венчурные инвесторы и основатели стартапов используют для этого трехступенчатую модель: TAM (Total Addressable Market) — общий доступный рынок, SAM (Serviceable Addressable Market) — обслуживаемый доступный рынок, и SOM (Serviceable Obtainable Market) — достижимый рынок[27]. Данный анализ позволяет перейти от абстрактных цифр о миллионах соискателей к конкретным, обоснованным количественным целям для бизнеса.

TAM представляет собой максимально возможный объем выручки, которую мог бы генерировать продукт при стопроцентном проникновении на рынок, без учета ограничений по географии, каналам сбыта или особенностям продукта. Для платформы Resugen целевым является рынок цифровых сервисов для соискателей в Российской Федерации, которые потенциально

могут быть заинтересованы в технологичных инструментах для улучшения своих карьерных документов.

В качестве отправной точки можно взять данные о численности экономически активного населения России. По данным Росстата на 2024 год, эта цифра составляет около 69 процентам, а это примерно 100 миллионов человек[28]. Однако далеко не все из них ежегодно занимаются активным поиском работы и готовы платить за соответствующие сервисы. Более релевантной группой являются люди, совершающие карьерные переходы: меняющие работу, ищущие первую работу после окончания учебы или находящиеся в активном поиске. Согласно научно-исследовательскому проекту цифровой аналитики вакансий Rutrud, доля таких «активных соискателей» в экономически активном населении за февраль 2026 года примерно 700000 человек, исходя из информации об уникальных резюме[29].

Среднее время поиска работы составляет около 2 месяцев[30]. Это означает, что за год через рынок проходит не одна и та же группа людей, а примерно 6 последовательных «слоёв» соискателей. Вычисляя по Формуле 3.3.1, получается примерно 4200000 уникальных соискателей в год.

$$TAM (\text{чел.}/\text{год}) = \text{уникальные соискатели в месяц} \cdot \left(\frac{12 \text{ месяцев}}{\text{Средний срок поиска}} \right) \quad (3.3.1)$$

В денежном выражении, если предположить среднегодовую выручку с одного платящего пользователя (ARPA) на уровне 780 рублей (эквивалент подписки 390 руб./мес. с учетом среднего времени поиска работы в 2 месяца), то общий потенциальный объем рынка (TAM) составляет примерно 3.28 миллиардов рублей в год. Эта оценка является теоретическим максимумом и показывает верхнюю границу возможностей бизнеса в национальном масштабе.

SAM представляет собой сегмент TAM, на который компания может реально выйти со своим продуктом, учитывая его особенности, ценовую

политику, каналы дистрибуции и текущие операционные ограничения. Это рынок, который географически и концептуально достижим для стартапа на текущем этапе.

Первым фильтром для сужения TAM до SAM является технологический и продуктовый фокус. Платформа Resugen изначально заточена под глубокий анализ и оптимизацию для ATS (Applicant Tracking Systems). Следовательно, ее основная ценность очевидна для соискателей, которые целенаправленно откликаются на вакансии в компаниях, использующих подобные системы. Это, в первую очередь, средний и крупный бизнес, международные корпорации, а также рекрутинговые агентства. Соискатели, ищущие работу в малом бизнесе, на фрилансе или через прямые знакомства, в меньшей степени являются целевой аудиторией на старте. Это сокращает целевую группу примерно на 30-40%.

Второй фильтр — лингвистический и региональный. На этапе MVP и в первые годы развития продукт будет полностью заточен под русский язык и специфику российского рынка труда (особенности hh.ru, 1С-платформ). Это исключает из SAM русскоязычных соискателей за рубежом, ориентированных на иностранные рынки, и сужает фокус до территории РФ.

Третий фильтр — каналы привлечения и осведомленность. Стартап не сможет мгновенно охватить всех потенциальных пользователей в силу ограниченности маркетингового бюджета. Первоначально привлечение будет идти через цифровые каналы (поиск, контекстная реклама, контент-маркетинг), которые наиболее эффективны для технологически продвинутой аудитории.

С учетом этих ограничений, SAM можно оценить как 20-30% от TAM. Это аудитория, которая не только теоретически нуждается в продукте, но и может узнать о нем через имеющиеся каналы коммуникации. В денежном выражении при том же ARPA в 780 рублей, SAM составляет 0.66 — 0.98 миллиардов рублей в год. Эта цифра демонстрирует реалистичный, но все

еще амбициозный рыночный потенциал для компании, которая сможет стать лидером в своей нише.

SOM — это часть SAM, которую компания реально планирует захватить в обозримой перспективе, обычно в первые 3-5 лет работы. Это наиболее консервативная и прагматичная оценка, которая учитывает наличие конкурентов, ресурсные ограничения стартапа, скорость роста и стратегию выхода на рынок. SOM является основой для построения финансовых прогнозов и постановки конкретных бизнес-целей.

Расчет SOM начинается с анализа доли рынка, которую новый игрок может разумно отвоевать. Несмотря на выявленную нишу, на рынке присутствуют конкуренты: бесплатный конструктор hh.ru (доминирующий игрок с максимальным охватом), платные нишевые сервисы вроде CareerLab, а также общая тенденция к использованию коммерческих LLM (YandexGPT, GigaChat и другие). Ключевой задачей является переключение пользователей с бесплатного, но неинтеллектуального решения hh.ru на платный, но дающий измеримую пользу продукт. В условиях SaaS-рынка захват 5-10% от SAM за первые 3-4 года считается отличным результатом для умелого стартапа с сильным продуктом.

Далее учитывается поэтапность выхода. В первый год фокус будет на узкой, но высокоплатежеспособной группе — IT-специалистах. Во второй год планируется расширение на смежные области (финансы, инженерия, digital-маркетинг). Только к третьему-четвертому году можно говорить о широком охвате всей аудитории SAM.

Исходя из этого, реалистичной целью на третий год работы можно считать привлечение 2-3% от SAM. В абсолютных цифрах это 20 000 — 30 000 платящих пользователей. При ARPA в 780 рублей, целевой объем SOM на третий год составляет 15.6 – 23.4 миллионов рублей годовой выручки (ARR). Для первого года работы цель будет скромнее — привлечение первых 3000-4500 платящих пользователей и достижение выручки в 2.34 – 3.51 миллионов рублей.

На Рисунке 3.3.2 предоставлены диаграммы по результатам проведенного анализа.

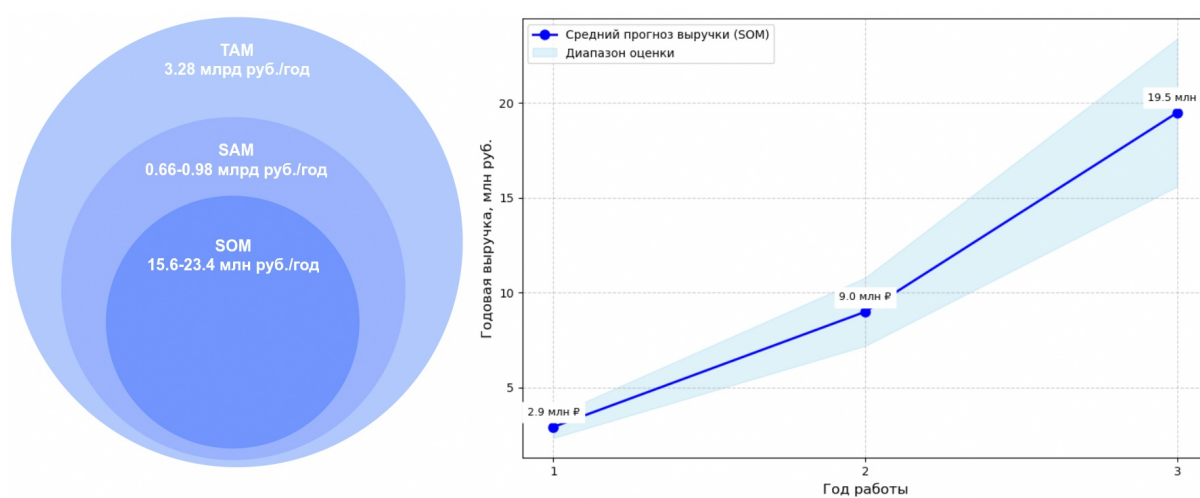


Рисунок 3.3.2 — Результаты анализа TAM, SAM, SOM

ЗАКЛЮЧЕНИЕ

В результате выполнения дипломной работы была достигнута поставленная цель: разработана архитектура и создан минимально жизнеспособный продукт (MVP) интеллектуальной веб-платформы «Resugen», выступающей в роли «цифрового карьерного ассистента». Решение позволяет соискателям адаптировать свои резюме к современным алгоритмическим системам отбора персонала за счёт автоматизированного анализа требований вакансий и интеллектуальной генерации структурированного контента. В ходе работы решены все сформулированные задачи:

1. Изучены международные и российские сервисы для создания и оптимизации резюме (включая конструкторы, ATS-ориентированные инструменты и платформы с ИИ-ассистентами). Выявлены их ключевые недостатки: отсутствие глубокой семантической привязки к конкретной вакансии, слабая поддержка совместимости с ATS и недостаточная прозрачность критериев оценки.
2. Детально описаны сценарии использования платформы: загрузка вакансии, анализ резюме, рекомендации по улучшению, генерация адаптированной версии документа. Определены нефункциональные требования (производительность, масштабируемость, безопасность данных, удобство интерфейса), что обеспечило системный подход к разработке.
3. Разработана микросервисная архитектура, включающая сервисы основного бизнес-процесса и аналитики.
4. Реализованы сервисы бизнес-процесса и фронтенд-часть.
5. Разработана модель Freemium с базовым бесплатным функционалом и платными подписками и произведена оценка рыночного потенциала.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The Evolution of Applicant Tracking System: A Historical Perspective // OnBlick URL: <https://www.onblick.com/blogs/the-evolution-of-applicant-tracking-system-a-historical-perspective> (дата обращения: 18.04.2026).
2. Naoki Otani, Nikita Bhutani, Estevam Hruschka, Natural Language Processing for Human Resources: A Survey // Aclanthology. - 2025. - С. 583–597.
3. АТС система для ускорения подбора персонала: как технологии меняют рекрутинг в 2026 году // VC.ru URL: <https://vc.ru/hr/2787930-ats-sistema-dlya-podbora-personala> (дата обращения: 18.04.2026).
4. ИИ в скрининге резюме и подборе кандидатов: полный гид // 1SELLER URL: <https://1seller.ru/ii-v-skrininge-rezyume-i-podbore-kandidatov-polnyj-gid> (дата обращения: 18.04.2026).
5. Enhancing recruitment efficiency: An advanced Applicant Tracking System (ATS) // ResearchGate URL: https://www.researchgate.net/publication/382099903_Enhancing_recruitment_efficiency_An_advanced_Applicant_Tracking_System_ATS (дата обращения: 18.04.2026).
6. ИИ против горы резюме: как научить модель ранжировать кандидатов на данных вашей компании // HR Завтра URL: <https://naimee.ai/hrzavtra/ii-rezyume-obuchenie-modeli-ranzhirovat-kandidatov> (дата обращения: 18.04.2026).
7. Лучшие AI-решения для HR и подбора персонала в 2025 году: полный гид по технологиям искусственного интеллекта // VC.ru URL: <https://vc.ru/hr/2689003-ai-resheniya-dlya-hr-i-podbora-personala> (дата обращения: 18.04.2026).
8. How AI Chatbots Are Transforming Recruitment in 2026 // assess candidates URL: <https://www.assesscandidates.com/ai-chatbots-for-recruitment/> (дата обращения: 18.04.2026).

9. Suraj Francis Noronha, Aruna Doreen Manezes, Poornima shetty, V. S. Shrishma Rao Evolving Human Resource Management: Admitting the Power of Artificial Intelligence // ATLANTIS PRESS. - 2025. - С. 270-281.
10. Jane Castleman, Zeyu Shen, Blossom Metevier, Max Springer, Aleksandra Korolova Measuring Validity in LLM-based Resume Screening // ATLANTIS PRESS. - 2026
11. 15 лучших генераторов резюме на основе ИИ в 2026 году (обзор бесплатных и платных инструментов) // Autoppt URL: <https://autoppt.com/ru/blog/best-ai-resume-generators/> (дата обращения: 18.04.2026).
12. ИИ отсеивает большую часть соискателей: 75% резюме никогда не попадают к рекрутеру // ОНТ URL: <https://ont.by/ru/society-ru/view/ii-otseivaet-bolshuju-chast-soiskatelej-75-rezjume-nikogda-ne-popadajut-k-rekruteru-315277-2026> (дата обращения: 18.04.2026).
13. Что делать, если меня не берут на работу из-за ИИ-алгоритмов? // Т-Ж URL: <https://t-j.ru/ii-v-naime/> (дата обращения: 18.04.2026).
14. Системы скрининга резюме. Как адаптировать резюме к ATS чтобы получить работу? // VC.ru URL: <https://vc.ru/hr/1356442-sistemy-skrininga-rezyume-kak-adaptirovat-rezyume-k-ats-chtoby-poluchit-rabotu> (дата обращения: 18.04.2026). (В6) Мнение: соискателям нужно прекратить использовать ИИ для резюме и тестовых заданий // Хабр URL: <https://habr.com/ru/news/1002922/> (дата обращения: 18.04.2026).
15. Создание резюме для ИИ: ключевые моменты и роль помощника по подбору персонала ONO AI // ONO URL: <https://onenewone.com/blog-ru/Создание-резюме-для-ИИ-ключевые-момен> (дата обращения: 18.04.2026).
16. Мнение: соискателям нужно прекратить использовать ИИ для резюме и тестовых заданий // Хабр URL: <https://habr.com/ru/news/1002922/> (дата обращения: 18.04.2026).

17. ATS для HR в 2025. Что происходит в России и мире — и к чему готовиться бизнесу // HR Завтра URL: <https://naimee.ai/hrzavtra/ats-dlya-hr-v-2025-perspektivy-i-trendy> (дата обращения: 18.04.2026).
18. Snehal Suryawanshi, Prajwal Mali, Sarthak Rasal Resume parsing and job recommendation using NLP and Machine learning // IJRTI. - 2025. - № 2456-3315
19. Екатерина Субботина, Александр Анисимов, Мария Засканова Открываем новые звезды автоматизации рекрутмента на российском рынке вместе с ТеДо // Хантфлоу. - 2025
20. Лучшие нейросети для создания резюме в 2025 году // VC.ru URL: <https://vc.ru/hr/2325297-luchshie-neurosseti-dlya-sozdaniya-rezyume> (дата обращения: 18.04.2026).
21. ТОП-8 лучших сервисов для создания профессионального резюме в 2025 году // VC.ru URL: <https://vc.ru/hr/2614507-luchshie-servisy-dlya-sozdaniya-rezyume> (дата обращения: 18.04.2026).
22. Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Zhouchi Lin, Bowen Zhang, Lionel Ni, Wen Gao, Yuanzhuo Wang, Jian Guo A survey on LLM-as-a-judge // ScienceDirect. - 2025
23. Frank P.-W. Lo, Jianing Qiu, Zeyu Wang, Haibao Yu, Yeming Chen, Gao Zhang, Benny Lo AI Hiring with LLMs: A Context-Aware and Explainable Multi-Agent Framework for Resume Screening // arxiv. - 2025
24. Ryan Lail An Empirical Investigation of Practical LLM-as-a-Judge Improvement Techniques on RewardBench 2 // arxiv. - 2026
25. Vidhi Agrawal From Freemium To Enterprise: Rethinking SaaS Pricing // arxiv. - 2025
26. ФАС признала hh.ru доминирующим на рынке онлайн-рекрутинга // NEW-RETAIL URL: https://new-retail.ru/novosti/retail/fas_priznala_hh_ru_dominiruyushchim_na_rynke_onlayn_rekrutinga/ (дата обращения: 18.04.2026).

27. Определить TAM SAM SOM // Podbiratel URL:
<https://podbiratel.com/pm/market-ideas-analysis/tam-sam-som> (дата обращения: 18.04.2026).
28. Уровень экономически активного населения в Российской Федерации // iMonitoring URL: <https://www.iminfin.ru/news/584-uroven-ekonomicheskii-aktivnogo-naseleniya-v-rossijskoj-federatsii> (дата обращения: 18.04.2026).
29. Краткий отчет по рынку труда за февраль 2026 // rutrud.com URL:
<https://rutrud.com/2026-02/> (дата обращения: 18.04.2026).
30. Какой срок поиска работы считается нормальным? // Т-Ж URL:
<https://t-j.ru/job-search-duration/> (дата обращения: 18.04.2026).